



## **Module 2 — Advanced SQL**

Master performance-focused SQL techniques — including Window Functions, CTEs, Advanced Joins, Advanced Subqueries, Query Optimization, Indexing basics, and Advanced Aggregations such as Ranking, Running Totals, and Moving Averages used.

**Author  
Srinivas Dande**





## Module 2 — Advanced SQL

### 1. Handling NULLs in Aggregates

- 1.1. How null works with Aggregates
- 1.2. Practice Lab – Handling NULLs in Aggregates
- 1.3. Query Examples
- 1.4. Key Takeaways

### 2. Advanced Joins

- 2.1. Practice Lab - Advanced Joins
- 2.2. NATURAL Join
- 2.3. USING() Clause
- 2.4. N-Table JOIN with FILTER Condition

### 3. Advanced Subqueries

- 3.1. Practice Lab - Advanced Subqueries
- 3.2. Scalar Subqueries
- 3.3. Scalar Subquery vs JOIN (Performance & Use Case)
- 3.4. Correlated Subqueries
- 3.5. Correlated vs Independent Subquery
- 3.6. EXISTS / NOT EXISTS & Performance
- 3.7. EXISTS vs IN
- 3.8. EXISTS vs JOIN



## 4. Joins vs Subqueries

- 4.1. Quick Comparison
- 4.2. Example — Using JOIN
- 4.3. Example — Using Subquery
- 4.4. Example — JOIN vs Scalar Subquery
- 4.5. Example — Nested Subquery vs Multi-Table JOIN
- 4.6. Example — Correlated Subquery vs JOIN
- 4.7. Execution Flow
- 4.8. When to Use Which
- 4.9. Best Practices

## 5. Common Table Expressions (CTEs)

- 5.1. What are CTEs?
- 5.2. Simple CTE Example
- 5.3. CTE with Joins Inside
- 5.4. Multiple CTEs
- 5.5. Nested (Chained) CTEs
- 5.6. Recursive CTE
- 5.7. Benefits of Using CTEs
- 5.8. Best Practices



## 6. Window Functions

- 6.1. What are Window Functions?
- 6.2. Practice Lab - Window Functions:
- 6.3. ROW\_NUMBER()
- 6.4. RANK() and DENSE\_RANK()
- 6.5. NTILE(n)
- 6.6. LAG() & LEAD()
- 6.7. Running Total (Cumulative SUM)
- 6.8. Moving Total & Moving Average (Rolling Windows)
- 6.9. Frame Clauses (ROWS BETWEEN...)
- 6.10. Percentile Functions
- 6.11. FIRST\_VALUE() & LAST\_VALUE()
- 6.12. Running Percentage (% Contribution)

## 7. SQL Execution Order

- 7.1. Written Order vs Actual Execution Order
- 7.2. Step-by-Step Execution Meaning
- 7.3. Full Execution Example
- 7.4. Why Execution Order Matters

## 8) Optimization & Performance Insights

- 8.1. Query Optimization Basics
- 8.2. How Indexes Work
- 8.3. Query Execution Plan (EXPLAIN)
- 8.4. Performance Optimization Techniques
- 8.5. Intro to Query Caching



## 1. Handling NULLs in Aggregates

- 1.1. How null works with Aggregates
- 1.2. Practice Lab – Handling NULLs in Aggregates
- 1.3. Query Examples
- 1.4. Key Takeaways



## 1.1. How NULL Works with Aggregates

- Aggregate functions treat NULLs differently:
- Functions that ignore NULLs
  - ✓ SUM(column)
  - ✓ AVG(column)
  - ✓ MIN(column)
  - ✓ MAX(column)
  - ✓ COUNT(column) → counts only non-NULL values
- Function that includes NULL rows
  - ✓ COUNT(\*) → counts every row, even if all columns are NULL

## 1.2. Practice Lab – Handling NULLs in Aggregates

### ■ # Step 1: Create Table

```
CREATE TABLE mystudents (  
    sid INT PRIMARY KEY,  
    sname VARCHAR(20),  
    email VARCHAR(30),  
    phone BIGINT,  
    course VARCHAR(20),  
    bal DECIMAL(10,2)  
);
```



## ■ # Step 2: Insert Sample Records (with NULLs)

```
INSERT INTO mystudents VALUES
(101, 'Srinivas', 'sri@da.com', 9991112222, 'SQL', 12000),
(102, SD, 'ssd@da.com', NULL, 'SQL', 15000),
(103, DS, 'vas@da.com', 8885554444, 'Python', NULL),
(104, 'Ravi', 'ravi@da.com', 7776663333, 'Python', 8000),
(105, 'Harsha', 'harsha@da.com', NULL, 'AI/ML', 10000),
(106, 'Meena', 'meena@da.com', 9990001111, 'AI/ML', NULL),
(107, 'Arjun', 'arjun@da.com', 8880001111, 'DL', 20000),
(108, 'Divya', 'divya@da.com', NULL, 'DL', 0),
(109, 'Ramesh', 'ramesh@da.com', 9995556666, NULL, NULL);
```

### 1.3. Query Examples

#### -- Q1) COUNT(column) vs COUNT(\*)

```
SELECT
    COUNT(phone) AS valid_phones,
    COUNT(*) AS total_records
FROM mystudents;
```

# COUNT(phone) → counts only non-NULL phones.

# COUNT(\*) → counts every row.

#### -- Q2) Ignoring vs Replacing NULL in AVG

```
SELECT
    AVG(bal) AS avg_balance1,
    AVG(COALESCE(bal, 0)) AS avg_balance2
FROM mystudents;
```

# COALESCE(bal,0) lets you decide how to treat missing balances.



## -- Q3) Total & Average Balance per Course

```
SELECT
    course,
    SUM(COALESCE(bal, 0)) AS total_balance,
    AVG(COALESCE(bal, 0)) AS avg_balance
FROM mystudents
GROUP BY course;
```

# Includes courses even with NULL or zero balances.

# Avoids skipping students with missing data.

## -- Q4) Group by Course and Handle Missing Course as 'Unknown'

```
SELECT
    COALESCE(course, 'Unknown') AS course_name,
    COUNT(*) AS student_count
FROM mystudents
GROUP BY COALESCE(course, 'Unknown');
```

# Groups NULL courses under "Unknown".

# Ensures every student is counted.

## -- Q5) Count Students by Phone Availability

```
SELECT
    CASE
        WHEN phone IS NULL THEN 'No Phone'
        ELSE 'Has Phone'
    END AS phone_status,
    COUNT(*) AS student_count
FROM mystudents
GROUP BY phone_status;
```

# Shows how many students have or don't have phone numbers.

## -- Q6) Find Highest and Lowest Balance per Course

```
SELECT
    course,
    MAX(COALESCE(bal,0)) AS max_balance,
    MIN(COALESCE(bal,0)) AS min_balance
FROM mystudents
GROUP BY course;
```

# Replaces NULLs with 0 before comparison.

# Ensures every course is represented even if all balances are NULL.

## -- Q7) Percentage of Students with Phone per Course

```
SELECT
    course,
    ROUND( (COUNT(phone) / COUNT(*)) * 100 , 2) AS percent_with_phone
FROM mystudents
GROUP BY course;
```

```
SELECT
    COALESCE(course, 'Unknown') AS course,
    ROUND( (COUNT(phone) / COUNT(*)) * 100 , 2) AS percent_with_phone
FROM mystudents
GROUP BY COALESCE(course, 'Unknown');
```

# Key Logic

COUNT(phone) → counts only valid phone numbers

COUNT(\*) → total per course

Division gives percentage



## 1.4. Key Takeaways

- Aggregate Functions Ignore NULLs
- Except COUNT(\*),
  - ✓ NULLs do not participate in: SUM, AVG, MIN, MAX, COUNT(column)
- Use CASE or COALESCE in GROUP BY
  - ✓ To ensure NULLs appear as: "Unknown" or 0 or "Not Provided"
- **Interview Tip:**
  - ✓ COUNT(column) ignores NULL
  - ✓ COUNT(\*) counts all rows
  - ✓ COALESCE() controls how you handle NULL



## 2. Advanced Joins

2.1. Practice Lab - Advanced Joins

2.2. NATURAL Join

2.3. USING() Clause

2.4. N-Table JOIN with FILTER Condition



## 2.1. Practice Lab - Advanced Joins

### Step 1: Create Tables

```
CREATE TABLE customers (  
  cid INT(3) PRIMARY KEY,  
  cname CHAR(15) NOT NULL,  
  email CHAR(25) NOT NULL UNIQUE,  
  phone BIGINT NOT NULL UNIQUE  
);
```

```
CREATE TABLE accounts (  
  cid INT(3),  
  accno INT(5) PRIMARY KEY,  
  atype CHAR(2) NOT NULL,  
  bal DOUBLE NOT NULL  
);
```

```
CREATE TABLE address (  
  cid INT(3),  
  addid INT(3) PRIMARY KEY,  
  street CHAR(15) NOT NULL,  
  city CHAR(15) NOT NULL,  
  state CHAR(15) NOT NULL  
);
```

### Step 2: Insert Sample Records

-- Customers

```
INSERT INTO customers VALUES  
(101,'sri','sri@jlc',111),  
(102,'vas','vas@jlc',222),  
(103,'sd','sd@jlc',333),  
(104,'ds','ds@jlc',444),  
(105,'hello','hello@jlc',555),  
(106,'hai','hai@jlc',666),  
(107,'aaa','aaa@jlc',777),  
(108,'bbb','bbb@jlc',888),  
(109,'ccc','ccc@jlc',999);
```

-- Accounts

```
INSERT INTO accounts VALUES
(101,12345,'SA',5000),
(102,12346,'SA',15000),
(103,12347,'SA',25000),
(107,12348,'SA',3000),
(108,12349,'SA',13000),
(109,12350,'SA',18000);
```

-- Address

```
INSERT INTO address VALUES
(101,1,'BTM','Blore','KA'),
(102,2,'MHA','Blore','KA'),
(103,3,'P1','Pune','MH'),
(104,4,'D1','Delhi','Delhi'),
(109,5,'D2','Delhi','Delhi'),
(110,6,'P2','Pune','MH'),
(111,7,'H1','Hyd','TG'),
(112,8,'PP','Patna','BR');
```

## **2.2. NATURAL Join**

- NATURAL JOIN automatically joins two (or more) tables based on all columns having the same name and compatible data types.
- You don't write any ON condition — SQL matches them internally.
- It always performs an INNER JOIN (returns only matching rows).

### **NATURAL INNER JOIN**

**Syntax:**

```
SELECT columns
FROM table1
NATURAL JOIN table2;
```

## **NATURAL LEFT JOIN**

### **Syntax**

```
SELECT columns  
FROM table1  
NATURAL LEFT JOIN table2;
```

## **NATURAL RIGHT JOIN**

### **Syntax**

```
SELECT columns  
FROM table1  
NATURAL RIGHT JOIN table2;
```

## **NATURAL JOIN Examples**

### **-- Q1) Customers with their Accounts**

```
SELECT cname, email, accno, bal  
FROM customers  
NATURAL JOIN accounts;
```

### **-- Q2) Customers with their Address**

```
SELECT cname, email, city, state  
FROM customers  
NATURAL JOIN address;
```

### **-- Q3) Combine customers, accounts, and address**

```
SELECT cname, phone, accno, bal, city, state  
FROM customers  
NATURAL JOIN accounts  
NATURAL JOIN address;
```



## LEFT / RIGHT NATURAL JOIN Examples

### -- Q4) NATURAL LEFT JOIN — include all customers even if account missing

```
SELECT cname, email, accno, bal  
FROM customers  
NATURAL LEFT JOIN accounts;
```

### -- Q5) NATURAL RIGHT JOIN — include all accounts even if customer missing

```
SELECT cname, email, accno, bal  
FROM customers  
NATURAL RIGHT JOIN accounts;
```

## Advantages

- ✓ Short and simple syntax
- ✓ No need to manually write join condition
- ✓ Automatically merges duplicate join columns

## Limitations

- ✓ Joins using all same-named columns → can cause unexpected behavior
- ✓ Schema changes (adding a same-named column) can silently break queries
- ✓ Not readable in large production queries because join columns are hidden

## # Best Practice

- ✓ Use NATURAL JOIN only when:
  - Table structures are stable
  - Column names match intentionally
  - You want quick, clean syntax for small datasets
- ✓ Otherwise prefer USING() or ON for clarity in real systems for clarity and safety.



## 2.3. USING() Clause

- USING() Clause specifies which column(s) should be used for the join.
- It removes duplicate join columns from the result and is safer than NATURAL JOIN.

### **Syntax**

```
SELECT columns  
FROM table1  
[INNER | LEFT | RIGHT] JOIN table2  
USING(column_name);
```

### **A) INNER JOIN with USING()**

#### **-- Q1) Customers with Accounts**

```
SELECT cname, email, accno, bal  
FROM customers  
INNER JOIN accounts  
USING(cid);
```

#### **-- Q2) Customers with Address**

```
SELECT cname, email, city, state  
FROM customers  
INNER JOIN address  
USING(cid);
```

#### **-- Q3) Combine all three tables**

```
SELECT cname, phone, accno, bal, city, state  
FROM customers  
INNER JOIN accounts USING(cid)  
INNER JOIN address USING(cid);
```



## **B) LEFT JOIN with USING()**

**-- Q4) All customers, including those without accounts**

```
SELECT cname, accno, bal  
FROM customers  
LEFT JOIN accounts  
USING(cid);
```

**-- Q5) All customers, including those without address**

```
SELECT cname, city, state  
FROM customers  
LEFT JOIN address  
USING(cid);
```

**-- Q6) Combine all three (keep all customers)**

```
SELECT cname, phone, accno, bal, city, state  
FROM customers  
LEFT JOIN accounts USING(cid)  
LEFT JOIN address USING(cid);
```

## **C) RIGHT JOIN with USING()**

**-- Q7) All accounts, including those not linked to any customer**

```
SELECT cname, accno, bal  
FROM customers  
RIGHT JOIN accounts  
USING(cid);
```



**-- Q8) All addresses, including those without customers**

```
SELECT cname, city, state
FROM customers
RIGHT JOIN address
USING(cid);
```

**-- Q9) Combine all three (keep all address records)**

```
SELECT cname, phone, accno, bal, city, state
FROM customers
RIGHT JOIN accounts USING(cid)
RIGHT JOIN address USING(cid);
```

## **2.4. N-Table JOIN with FILTER Condition**

**-- Q1) Customers from Blore or Pune with balance >= 15000**

```
SELECT cname, phone, accno, bal, city, state
FROM customers
JOIN accounts USING(cid)
JOIN address USING(cid)
WHERE city IN ('Blore','Pune') AND bal >= 15000;
```

**-- Q2) Keep all customers (even if no account or address)**

```
SELECT c.cid, c.cname, a.accno, a.bal, ad.city, ad.state
FROM customers c
LEFT JOIN accounts a
USING (cid)
LEFT JOIN address ad
USING (cid);
```



-- Q3) Show all customers who live in Blore OR do not have any address record.

```
SELECT
    c.cid, c.cname, a.accno, a.bal, ad.city, ad.state
FROM customers c
LEFT JOIN accounts a USING (cid)
LEFT JOIN address ad USING (cid)
WHERE ad.city = 'Blore' OR ad.city IS NULL;
```

-- Q4) Customers in KA with 'SA' accounts and balance between 10k-25k

```
SELECT c.cname, a.accno, a.bal, ad.city, ad.state
FROM customers c
JOIN accounts a USING(cid)
JOIN address ad USING(cid)
WHERE ad.state = 'KA'
AND a.atype = 'SA'
AND a.bal BETWEEN 10000 AND 25000;
```

### Advantages:

- ✓ Much more readable than ON when column names are identical
- ✓ Automatically removes duplicate join column from output
- ✓ Works with all join types (INNER / LEFT / RIGHT )
- ✓ Prevents accidental multiple-column matches (unlike NATURAL JOIN)

### Limitations:

- ✓ Can be used only if both tables have the same column name
- ✓ If PK/FK column names differ → must use ON instead

### Best Practice:

- ✓ Rename your foreign key to match the primary key (like both cid) →
- ✓ Then use JOIN ... USING(cid) for simple, clean joins.



## 3. Advanced Subqueries

- 3.1. Practice Lab - Advanced Subqueries
- 3.2. Scalar Subqueries
- 3.3. Scalar Subquery vs JOIN
- 3.4. Correlated Subqueries
- 3.5. Correlated vs Independent Subquery
- 3.6. EXISTS / NOT EXISTS & Performance
- 3.7. EXISTS vs IN
- 3.8. EXISTS vs JOIN



## 3.1. Practice Lab - Advanced Subqueries

- We're continuing with the same dataset:

```
customers(cid, cname, email, phone)
accounts(cid, accno, atype, bal)
address(cid, addid, street, city, state)
```

## 3.2. Scalar Subqueries (in SELECT List)

- Scalar subquery returns a single value (one row, one column).
- Used inside a SELECT statement
- It behaves like a constant or computed field.
- It is executed once per outer row in the query.

### Syntax

```
SELECT column1,
      (SELECT single_value FROM table2 WHERE condition) AS alias
FROM table1;
```

### A) Single Value (One Row, One Column)

#### -- Q1) Display each customer with their account balance

```
SELECT cname,
      (SELECT bal FROM accounts a WHERE a.cid = c.cid) AS balance
FROM customers c;
```

#### # How It Works:

- Inner query finds bal from accounts where cid matches outer row.
- Runs once per customer.
- If no matching account → returns NULL.



## -- Q2) Display each customer with their city

```
SELECT cname,  
       (SELECT city FROM address ad WHERE ad.cid = c.cid) AS city  
FROM customers c;
```

### # How It works

- Inner query finds the city from address where cid matches the outer row.
- It runs once for each customer.
- If no matching address → returns NULL.

## B) Derived / Calculated Column

## -- Q3) Display each customer with their balance after adding 10% bonus

```
SELECT cname,  
       (SELECT bal * 1.10 FROM accounts a WHERE a.cid = c.cid) AS new_balance  
FROM customers c;
```

## -- Q4) Display customer with both balance and city using subqueries

```
SELECT cname,  
       (SELECT bal FROM accounts a WHERE a.cid = c.cid) AS balance,  
       (SELECT city FROM address ad WHERE ad.cid = c.cid) AS city  
FROM customers c;
```

### # Best Practices

- ✓ Always make sure the subquery returns one value → use MAX(), MIN(), or LIMIT 1 when needed.
- ✓ For better performance, replace scalar subqueries with JOINS or CTEs if large data volumes are involved.
- ✓ Perfect for small datasets, analytical columns, or when readability matters.



### 3.3. Scalar Subquery vs JOIN (Performance & Use Case)

- Both Scalar Subqueries and JOINS can fetch related data from another table,
- but they differ in execution flow, readability, and performance.

#### **Example: Customers with Their Account Balance**

##### **Approach 1 — Scalar Subquery**

```
SELECT cust.cname,  
       (SELECT acc.bal  
        FROM accounts acc  
        WHERE acc.cid = cust.cid) AS balance  
FROM customers cust;
```

##### **# How It works**

Inner query runs once for each row of the customers table.

For every customer, it looks up the corresponding balance from accounts.

If no account exists, balance will be NULL.

##### **Execution Flow:**

Outer query → pick one customer → run inner query → repeat for next customer.

##### **Approach 2 — JOIN**

```
SELECT c.cname, a.bal  
FROM customers c  
LEFT JOIN accounts a  
USING (cid);
```

##### **# How It works**

Join happens once — MySQL merges both tables based on cid.

Missing accounts → bal will show NULL.

##### **Execution Flow:**

Database engine merges both tables in one pass using indexes → faster.

## Performance Difference

| Factor        | Scalar Subquery                  | JOIN                                        |
|---------------|----------------------------------|---------------------------------------------|
| Execution     | Runs subquery once per outer row | Executes in a single pass                   |
| Performance   | Slower for large tables          | Faster and optimized                        |
| Readability   | Simpler for single lookups       | Best for multi-table combinations           |
| Null Handling | Returns NULL if no match         | Returns NULL (same behavior)                |
| Optimization  | Limited                          | Query optimizer can use indexes efficiently |

## When to Use Which

- |                                                 |                                      |
|-------------------------------------------------|--------------------------------------|
| 1) Small table, one related value               | - Scalar Subquery (simple, readable) |
| 2) Large tables or many joins                   | - JOIN (faster, scalable)            |
| 3) Need computed or derived field               | - Scalar Subquery                    |
| 4) Fetching related info across multiple tables | - JOIN                               |

## 3.4. Correlated Subqueries

- Correlated subquery is a subquery that depends on a value from the outer query.
- It executes once for every row of the outer query and is “correlated” through a shared column.
- It’s like a mini-query that runs repeatedly — once for each outer row — checking related conditions.

### Syntax

```

SELECT columns
FROM outer_table out
WHERE expression OPERATOR (
    SELECT aggregate(col)
    FROM inner_table in
    WHERE in.col = out.col
);

```

# Inner query references the outer table’s column, creating a correlation.



## -- Q1) Customers Having Balance > Average Balance

```
SELECT cust.cname, acc1.bal
FROM customers cust
JOIN accounts acc1 ON cust.cid = acc1.cid
WHERE acc1.bal > (SELECT AVG(bal) FROM accounts);
```

### # How It Works:

#### a) Inner Query (Independent Subquery)

- ✓ SELECT AVG(bal) FROM accounts
- ✓ Computes the average balance across all accounts
- ✓ Runs only once, because it does not depend on outer data

#### b) Outer Query

- ✓ Joins customers and accounts
- ✓ Retrieves each customer's name and balance

#### c) Comparison

- ✓ Each customer's balance (acc1.bal)
- ✓ is compared with the single average value returned by the inner query

#### d) Final Output

- ✓ Returns customers whose balance is greater than the overall average

### Note:

- ✓ This query is not correlated because the inner query doesn't reference any outer columns.
- ✓ It's an independent subquery — used here as a setup before true correlation examples.



## Q2) Customers Whose Balance Is Greater Than Their City's Average(Correlated Subquery)

```
SELECT cust.cname, add1.city, acc1.bal
FROM customers cust
JOIN accounts acc1 ON cust.cid = acc1.cid
JOIN address add1 ON cust.cid = add1.cid
WHERE acc1.bal > (
    SELECT AVG(acc2.bal)
    FROM accounts acc2
    JOIN address add2 ON acc2.cid = add2.cid
    WHERE add2.city = add1.city
);
```

### How It Works

#### a) Outer Query

- ✓ Joins customers, accounts, and address
- ✓ Gets each customer's city and balance

#### b) Inner Query (Correlated Subquery)

- ✓ Joins accounts and address
- ✓ Filters for the same city as the outer row: → add2.city = add1.city
- ✓ Calculates the average balance for that city: → AVG(acc2.bal)
- ✓ Runs once per city, not once overall

#### c) Correlation

- ✓ Inner query depends on outer row through add1.city
- ✓ This makes it a correlated subquery

#### d) Comparison

- ✓ Compares each customer's balance (acc1.bal)
- ✓ with the average balance of their city

#### e) Final Output

- ✓ Returns customers who have higher-than-average balance
- ✓ within their own city

### Key Point

- ✓ add2.city = add1.city is the link that makes this a correlated subquery.



## Q3) Customers Whose Balance Is Greater Than Their City's Average (Using JOINS)

```
SELECT cust.cname, add1.city, acc1.bal
FROM customers cust
JOIN accounts acc1 ON cust.cid = acc1.cid
JOIN address add1 ON cust.cid = add1.cid
WHERE acc1.bal > (
    SELECT AVG(acc2.bal)
    FROM accounts acc2
    JOIN address add2 ON acc2.cid = add2.cid
    WHERE add2.city = add1.city
);
```

### How It Works:

#### a) Inner Query (city\_avg)

- ✓ Joins accounts and address
- ✓ Groups data by city
- ✓ Calculates the average balance per city

#### b) Outer Query

- ✓ Joins customers, accounts, and address
- ✓ Gets each customer's name, city, and balance

#### c) Join Condition

- ✓ The outer result is joined with city\_avg:
- ✓ `add1.city = city_avg.city` → ensures customer is matched with their city's average
- ✓ `acc1.bal > city_avg.avg_bal` → keeps customers whose balance is greater than their city average

#### d) Filter Result

- ✓ Only customers with above-average balance in their city are retained

#### e) Final Output

- ✓ Shows customer name, city, and balance
- ✓ Only those who have balance greater than their city's average

## -- Q4) Customers Having the Maximum Balance in Their City (Correlated Subquery)

```
SELECT cust.cname, add1.city, acc1.bal
FROM customers cust
JOIN accounts acc1 ON cust.cid = acc1.cid
JOIN address add1 ON cust.cid = add1.cid
WHERE acc1.bal = (
    SELECT MAX(acc2.bal)
    FROM accounts acc2
    JOIN address add2 ON acc2.cid = add2.cid
    WHERE add2.city = add1.city
);
```

### # How It Works:

#### a) Outer Query

- ✓ Joins customers, accounts, and address
- ✓ Gets each customer's name, city, and balance

#### b) Inner Query (Correlated Subquery)

- ✓ Joins accounts (acc2) with address (add2)
- ✓ Filters by the same city as the outer query: → add2.city = add1.city
- ✓ Calculates the maximum balance in that specific city: → MAX(acc2.bal)

#### c) Correlation

- ✓ Inner query uses a value from the outer query (add1.city)
- ✓ This makes it a correlated subquery
- ✓ It runs once per outer row

#### d) Comparison

- ✓ The outer customer's balance acc1.bal
- ✓ is compared with
- ✓ the maximum balance returned by the subquery

#### e) Final Output

- ✓ Returns only those customers
- ✓ whose balance = maximum balance of their city

## -- Q5) Customers Having the Maximum Balance in Their City (Using JOINS)

```
SELECT add1.city, cust.cname, acc1.bal
FROM customers cust
JOIN accounts acc1 ON cust.cid = acc1.cid
JOIN address add1 ON cust.cid = add1.cid
JOIN (
    SELECT add2.city, MAX(acc2.bal) AS max_bal
    FROM accounts acc2
    JOIN address add2 ON acc2.cid = add2.cid
    GROUP BY add2.city
) AS city_max
ON add1.city = city_max.city
AND acc1.bal = city_max.max_bal;
```

### # How It Works

#### a) Inner Query (city\_max):

- ✓ Joins accounts (acc2) and address (add2) to group balances by city.
- ✓ Calculates the maximum balance per city using MAX(acc2.bal).

#### b) Outer Query:

- ✓ Joins customers (cust), accounts (acc1), and address (add1) to get each customer's city and balance.

#### c) Join Condition:

- ✓ Joins the outer result with the inner city\_max subquery
  - add1.city = city\_max.city (same city)
  - acc1.bal = city\_max.max\_bal (balance equals city's max).

#### d) Filter Result:

- ✓ Keeps only customers whose balance matches the maximum balance of their city.

#### e) Final Output:

- ✓ Lists city, customer name, and balance of top-balance customers per city.



## Performance Notes

### a) Execution

- ✓ Correlated subquery runs once for each outer row.

### b) Performance

- ✓ Slow on large tables because the inner query repeats many times.

### c) Optimization

- ✓ SQL optimizers struggle to speed up deeply correlated queries.

### d) Better Alternative

- ✓ Rewrite using JOIN + GROUP BY so aggregation runs once.

### e) Indexing

- ✓ Always index correlated columns like cid, city to improve speed.

## Best Practices

### When to Use Correlated Subqueries

- ✓ When comparing each row with a related value (e.g., balance vs city average)
- ✓ When using EXISTS / NOT EXISTS for logical checks

### When to Avoid

- ✓ On large datasets
- ✓ When aggregate values can be computed once (use JOIN/CTE instead)

### Always Do

- ✓ Add indexes on correlation columns (cid, city)
- ✓ Keep correlation shallow (avoid nesting correlated subqueries)



## 3.5. Correlated vs Independent Subquery

### a) Independent Subquery — Customers with Balance > Global Average

```
SELECT cust.cname, acc1.bal
FROM customers cust
JOIN accounts acc1 ON cust.cid = acc1.cid
WHERE acc1.bal > (SELECT AVG(bal) FROM accounts);
```

#### # How It Works:

- ✓ Inner query → returns one constant value (global average).
- ✓ Outer query → compares all customers to that single value.
- ✓ Executes once → faster.

### b) Correlated Subquery — Customers with Balance > City Average

```
SELECT cust.cname, add1.city, acc1.bal
FROM customers cust
JOIN accounts acc1 ON cust.cid = acc1.cid
JOIN address add1 ON cust.cid = add1.cid
WHERE acc1.bal > (
    SELECT AVG(acc2.bal)
    FROM accounts acc2
    JOIN address add2 ON acc2.cid = add2.cid
    WHERE add2.city = add1.city
);
```

#### # How It Works:

- ✓ Inner query → recalculates city average for each customer's city.
- ✓ Executes once per outer row → slower but more specific.
- ✓ Correlation happens via add2.city = add1.city.



## Independent SubQuery Vs Correlated SubQuery

| Feature                | Independent SubQuery      | Correlated SubQuery    |
|------------------------|---------------------------|------------------------|
| Inner Query Runs       | Once                      | For every outer row    |
| Depends on Outer Query | No                        | Yes                    |
| Performance            | Fast                      | Slower                 |
| Typical Use            | Global comparison         | Row/group comparison   |
| Examples               | Balance > overall average | Balance > city average |

### Interview Tip

- ✓ Correlated subquery runs once per outer row.
- ✓ Good for row-level checks, but slower on large data.
- ✓ Use JOIN + GROUP BY for better performance.

## 3.6. EXISTS / NOT EXISTS & Performance

### a) EXISTS

- ✓ EXISTS checks whether a subquery returns at least one row.
- ✓ If any row found → condition is TRUE.
- ✓ If zero rows → condition is FALSE.

### b) NOT EXISTS

- ✓ NOT EXISTS returns TRUE when the subquery returns no rows.
- ✓ If zero rows → TRUE
- ✓ If any row found → FALSE

These are mainly used with correlated subqueries to test if related data exists or not.

**Syntax**

```
SELECT columns
FROM table1 t1
WHERE EXISTS (
    SELECT 1
    FROM table2 t2
    WHERE t2.col = t1.col
);
```

**Key Note:**

- ✓ SELECT 1 part is a convention —
- ✓ We don't need actual data, only the existence of rows matters.

**Q1) Customers Having an Account**

```
SELECT cust.cname
FROM customers cust
WHERE EXISTS (
    SELECT 1
    FROM accounts acc1
    WHERE acc1.cid = cust.cid
);
```

**How It Works**

- ✓ Outer Query: Reads each customer.
- ✓ Inner Query: Checks accounts for matching cid.
- ✓ Condition: If a matching account exists → TRUE.
- ✓ Final Output: Customers who have an account.

**Q2) Customers Without Any Account**

```
SELECT cust.cname
FROM customers cust
WHERE NOT EXISTS (
    SELECT 1
    FROM accounts acc1
    WHERE acc1.cid = cust.cid
);
```



## How It Works

- ✓ Outer Query: Reads each customer.
- ✓ Inner Query: Checks accounts for same cid.
- ✓ Condition: If no account found → TRUE.
- ✓ Final Output: Customers who don't have any account.

### Q3) Customers Living in 'Blore' Having an Account

```
SELECT cust.cname, cust.email, cust.phone
FROM customers cust
WHERE EXISTS (
    SELECT 1
    FROM accounts acc1
    WHERE acc1.cid = cust.cid
)
AND EXISTS (
    SELECT 1
    FROM address add1
    WHERE add1.cid = cust.cid
    AND add1.city = 'Blore'
);
```

## How It Works

- ✓ 1st EXISTS: Has account?
- ✓ 2nd EXISTS: Lives in Blore?
- ✓ Condition: Both must be TRUE.
- ✓ Final Output: Customers who have an account AND live in Blore.

### Q4) Customers Having High Balance Accounts (> 15000)

```
SELECT cust.cname
FROM customers cust
WHERE EXISTS (
    SELECT 1
    FROM accounts acc1
    WHERE acc1.cid = cust.cid
    AND acc1.bal > 15000
);
```



## How It Works

- ✓ Inner Query: Looks for matching cid + balance > 15000.
- ✓ Condition: If found → TRUE.
- ✓ Final Output: Customers who have an account with balance > 15000.

## Q5) Customers Who Don't Have an Address

```
SELECT cust.cname
FROM customers cust
WHERE NOT EXISTS (
    SELECT 1
    FROM address add1
    WHERE add1.cid = cust.cid
);
```

### How It Works

- ✓ Inner Query: Looks for matching cid in address table.
- ✓ Condition: If no address exists → TRUE.
- ✓ Final Output: Customers without an address.

## Q6) Customers Living in 'Pune'

```
SELECT cust.cname, cust.email, cust.phone
FROM customers cust
WHERE EXISTS (
    SELECT 1
    FROM address add1
    WHERE add1.cid = cust.cid
    AND add1.city = 'Pune'
);
```

### How It Works

- ✓ Outer Query: Reads each customer.
- ✓ Inner Query: Searches address table for matching cid AND city = 'Pune'.
- ✓ Condition: If at least one address in Pune exists → TRUE.
- ✓ Final Output: Customers who live in Pune.



## 3.7. EXISTS vs IN

### Using IN

```
SELECT cust.cname
FROM customers cust
WHERE cust.cid IN (SELECT acc1.cid FROM accounts acc1);
```

### Using EXISTS

```
SELECT cust.cname
FROM customers cust
WHERE EXISTS (
    SELECT 1
    FROM accounts acc1
    WHERE acc1.cid = cust.cid
);
```

### Output

- ✓ Both return the same result → customers who have accounts.

### Performance

- ✓ **EXISTS** → usually faster on large tables (stops after finding the first match)
- ✓ **IN** → evaluates the entire subquery result (slow if large or contains NULLs)

### Comparison Table — EXISTS vs IN

| Aspect        | EXISTS                              | IN                                  |
|---------------|-------------------------------------|-------------------------------------|
| Behavior      | TRUE if any matching row exists     | Compares value with a list/subquery |
| Execution     | Stops after first match (optimized) | Scans full list/subquery            |
| NULL Handling | Ignores NULL safely                 | NULLs can cause missed matches      |
| Best Use Case | Large tables, correlated checks     | Small lists or small subqueries     |
| Performance   | Faster for large datasets           | Slower if many rows or NULLs exist  |



## 3.8. EXISTS vs JOIN

### Using JOIN

```
SELECT DISTINCT cust.cname
FROM customers cust
JOIN accounts acc1 ON cust.cid = acc1.cid;
```

### Using EXISTS

```
SELECT cust.cname
FROM customers cust
WHERE EXISTS (
    SELECT 1
    FROM accounts acc1
    WHERE acc1.cid = cust.cid
);
```

### Output

- ✓ Both return customers who have at least one account.

### Performance

- ✓ **JOIN** → use when you need data from both tables
- ✓ **EXISTS** → use when you only need to check whether a match exists

### Comparison Table — EXISTS vs JOIN

| Aspect      | EXISTS                                      | JOIN                                  |
|-------------|---------------------------------------------|---------------------------------------|
| Purpose     | Checks if a related row exists              | Combines rows from multiple tables    |
| Returns     | Only columns from outer table               | Columns from both tables              |
| Performance | Faster when only checking existence         | Better when retrieving data           |
| Duplicates  | Never returns duplicates (TRUE/FALSE logic) | May return duplicates unless DISTINCT |



## Performance Insights

### a) EXISTS

- ✓ Best for checking if related data exists (Fastest — stops after first match)

### b) NOT EXISTS

- ✓ Safe for “no match” checks (Handles NULL correctly)

### c) IN

- ✓ Use only for small, fixed lists

### d) NOT IN

- ✓ Avoid when subquery may return NULL (can produce incorrect results)

### e) Index Columns

- ✓ Index correlated columns (cid, city) for speed

### f) Short-Circuit Behavior

- ✓ EXISTS stops reading after first match → high performance

## Summary

| Operator   | TRUE When                     | Best Use                               |
|------------|-------------------------------|----------------------------------------|
| EXISTS     | Subquery returns $\geq 1$ row | Check if related record exists         |
| NOT EXISTS | Subquery returns 0 rows       | Check if related record does not exist |
| IN         | Value matches a list          | Small, static lists                    |
| NOT IN     | Value not in list             | Avoid if NULLs involved                |
| JOIN       | Rows matched & combined       | Retrieve data from both tables         |



## 4. Joins vs Subqueries

- Both JOINS and SUBQUERIES can pull data from multiple tables
- — but they differ in execution, readability, and performance.

### 4.1. Quick Comparison

### 4.2. Example — Using JOIN

### 4.3. Example —Using Subquery

### 4.4. Example — JOIN vs Scalar Subquery

### 4.5. Example — Nested Subquery vs Multi-Table JOIN

### 4.6. Example — Correlated Subquery vs JOIN

### 4.7. Execution Flow

### 4.8. When to Use Which

### 4.9. Best Practices



## 4.1. Quick Comparison

| Feature     | JOIN                                               | SUBQUERY                              |
|-------------|----------------------------------------------------|---------------------------------------|
| Definition  | Combines data from multiple tables into one result | Better for simple filtering           |
| Execution   | Processes all tables together                      | Inner query runs first (or per row)   |
| Performance | Faster (optimized by engine)                       | Slower (especially correlated)        |
| Readability | Better for combining related data                  | Better for simple filtering           |
| Use When    | You need columns from many tables                  | One query depends on another's result |

## 4.2. Example — Using JOIN

**Goal: Customers with accounts (show balance)**

```
SELECT cust.cname, acc.accno, acc.bal  
FROM customers cust  
JOIN accounts acc ON cust.cid = acc.cid;
```

### How It Works

- ✓ SQL joins rows where cust.cid = acc.cid
- ✓ Returns combined customer + account data in one result

## 4.3. Example —Using Subquery

**Goal: Same output but using subquery**

```
SELECT cust.cname  
FROM customers cust  
WHERE cust.cid IN (SELECT acc.cid FROM accounts acc);
```

### How It Works

- ✓ Inner query gets all cids from accounts
- ✓ Outer query returns customers whose cid appears inside that list



## 4.4. Example — JOIN vs Scalar Subquery

**Goal: Show each customer's balance (two methods)**

### **JOIN Version**

```
SELECT cust.cname, acc1.bal
FROM customers cust
JOIN accounts acc1 ON cust.cid = acc1.cid;
```

### **Scalar Subquery Version**

```
SELECT cust.cname,
       (SELECT acc1.bal FROM accounts acc1 WHERE acc1.cid = cust.cid) AS balance
FROM customers cust;
```

### **Performance**

| Method          | Execution    | Speed  |
|-----------------|--------------|--------|
| JOIN            | Single plan  | Fast   |
| Scalar Subquery | Runs per row | Slower |

## 4.5. Example — Nested Subquery vs Multi-Table JOIN

**Goal: Customers living in cities where at least one person has balance > 15000.**

### **Nested Subquery**

```
SELECT cust.cname, add1.city
FROM customers cust
JOIN address add1 ON cust.cid = add1.cid
WHERE add1.city IN (
    SELECT add2.city
    FROM address add2
    JOIN accounts acc2 ON add2.cid = acc2.cid
    WHERE acc2.bal > 15000
);
```



## Equivalent JOIN Version

```
SELECT DISTINCT cust.cname, add1.city
FROM customers cust
JOIN accounts acc1 ON cust.cid = acc1.cid
JOIN address add1 ON cust.cid = add1.cid
WHERE acc1.bal > 15000;
```

## Performance

JOIN is faster — avoids nested lookups.

## 4.6. Example — Correlated Subquery vs JOIN

**Goal: Customers whose balance is > their city average.**

### Correlated Subquery

```
SELECT cust.cname, add1.city, acc1.bal
FROM customers cust
JOIN accounts acc1 ON cust.cid = acc1.cid
JOIN address add1 ON cust.cid = add1.cid
WHERE acc1.bal > (
    SELECT AVG(acc2.bal)
    FROM accounts acc2
    JOIN address add2 ON acc2.cid = add2.cid
    WHERE add2.city = add1.city
);
```

### JOIN + GROUP BY

```
SELECT cust.cname, add1.city, acc1.bal
FROM customers cust
JOIN accounts acc1 ON cust.cid = acc1.cid
JOIN address add1 ON cust.cid = add1.cid
JOIN (
    SELECT add2.city, AVG(acc2.bal) AS avg_bal
    FROM accounts acc2
    JOIN address add2 ON acc2.cid = add2.cid
    GROUP BY add2.city
) AS city_avg
ON add1.city = city_avg.city
WHERE acc1.bal > city_avg.avg_bal;
```



## Performance Tip

Correlated → slower (runs per row)

JOIN → faster and scalable

## 4.7. Execution Flow

| Step | JOIN Flow                 | SUBQUERY Flow                      |
|------|---------------------------|------------------------------------|
| 1    | Reads all tables together | Executes inner query first         |
| 2    | Merges matching rows      | Passes inner result to outer query |
| 3    | Applies filters           | Outer query compares / filters     |
| 4    | Returns combined rows     | May re-run subquery for each row   |

## 4.8. When to Use Which

| Use Case                            | Best Choice         | Why                               |
|-------------------------------------|---------------------|-----------------------------------|
| Combine data from multiple tables   | JOIN                | Optimized, returns merged columns |
| Check if related row exists         | EXISTS              | Stops early, fast                 |
| Return a single derived value       | Scalar Subquery     | Simple & readable                 |
| Compare row vs group (city average) | Correlated Subquery | Logical structure                 |
| Large dataset analytics             | JOIN / CTE          | Engine optimizes JOINS better     |

## 4.9. Best Practices

- ✓ Prefer JOINS for combining data
- ✓ Prefer EXISTS over IN for large subqueries
- ✓ Replace correlated subqueries with JOIN + GROUP BY when possible
- ✓ Always index join/correlated columns (cid, city)
- ✓ Keep subqueries short and readable



## 5. Common Table Expressions (CTEs)

5.1. What are CTEs?

5.2. Simple CTE Example

5.3. CTE with Joins Inside

5.4. Multiple CTEs

5.5. Nested (Chained) CTEs

5.6. Recursive CTE

5.7. Benefits of Using CTEs

5.8. Best Practices



## 5.1. What are CTEs?

- CTE - Common Table Expression
- CTE is a temporary named result set that you can reference within a SELECT, INSERT, UPDATE, or DELETE statement.
- It improves readability, modularity, and allows step-by-step query building — just like variables in programming.

### **Syntax**

```
WITH CTE_name AS (  
    SELECT column_list  
    FROM table_name  
    WHERE condition  
)  
SELECT *  
FROM CTE_name;
```

- ✓ WITH clause defines the CTE.
- ✓ CTE exists only for that query (not stored permanently).
- ✓ You can reference it multiple times in the same query.



## 5.2. Simple CTE Example

-- Q1) Display customers with their balances using a CTE

```
WITH cust_bal_cte AS (  
    SELECT cid, bal  
    FROM accounts  
)  
SELECT cust.cname, cust.email, cb_cte.bal  
FROM customers cust  
JOIN cust_bal_cte cb_cte ON cust.cid = cb_cte.cid;
```

### How It Works

- ✓ CTE **cust\_bal\_cte** fetches all customer IDs and their balances from the accounts table.
- ✓ Main query joins **cust\_bal\_cte** with the customers table using cid.
- ✓ Finally, it displays each customer's name, email, and balance.

-- Q2) Customers having balance > 15000

```
WITH high_bal_cte AS (  
    SELECT cid, bal  
    FROM accounts  
    WHERE bal > 15000  
)  
SELECT c.cname, c.email, hb_cte.bal  
FROM customers c  
JOIN high_bal_cte hb_cte ON c.cid = hb_cte.cid;
```

### How It Works

- ✓ CTE **high\_bal\_cte** selects only customers whose balance is greater than 15000 from the accounts table.
- ✓ Main query joins customers with **high\_bal\_cte** using cid.
- ✓ Finally, it displays the customer's name, email, and balance for those high-value accounts.



## 5.3. CTE with Joins Inside

-- Q3) Customers along with their city and balance

```
WITH cust_info_cte AS (  
    SELECT c.cid, c.cname, a.bal, ad.city  
    FROM customers c  
    JOIN accounts a ON c.cid = a.cid  
    JOIN address ad ON c.cid = ad.cid  
)  
SELECT cname, city, bal  
FROM cust_info_cte;
```

### How It Works

- ✓ CTE cust\_info\_cte joins customers, accounts, and address tables to create a combined dataset.
- ✓ It includes each customer's ID, name, city, and balance.
- ✓ Main query simply selects data from this temporary result set — no need to repeat joins.



## 5.4. Multiple CTEs

-- Q4) Using two CTEs to calculate average and filter results

```
WITH avg_bal_cte AS (  
    SELECT AVG(bal) AS avg_balance  
    FROM accounts  
)  
,  
cust_bal_cte AS (  
    SELECT c.cname, a.bal  
    FROM customers c  
    JOIN accounts a ON c.cid = a.cid  
)  
SELECT cname, bal  
FROM cust_bal_cte, avg_bal_cte  
WHERE cust_bal_cte.bal > avg_bal_cte.avg_balance;
```

### How It Works

- ✓ avg\_bal\_cte calculates the overall average balance from all accounts.
- ✓ cust\_bal\_cte prepares each customer's name and balance.
- ✓ Main query compares every customer's balance with the average balance and returns only those above the average.



## 5.5. Nested (Chained) CTEs

-- Q5) Find customers who live in “Blore” and have balance > city average

```
WITH city_avg_cte AS (  
    SELECT ad.city, AVG(a.bal) AS avg_bal  
    FROM accounts a  
    JOIN address ad ON a.cid = ad.cid  
    GROUP BY ad.city  
)  
,  
cust_info_cte AS (  
    SELECT c.cname, ad.city, a.bal  
    FROM customers c  
    JOIN accounts a ON c.cid = a.cid  
    JOIN address ad ON c.cid = ad.cid  
)  
SELECT ci.cname, ci.city, ci.bal  
FROM cust_info_cte ci  
JOIN city_avg_cte ca ON ci.city = ca.city  
WHERE ci.bal > ca.avg_bal  
AND ci.city = 'Blore';
```

### How It Works

- ✓ city\_avg\_cte → Calculates the average balance per city.
- ✓ cust\_info\_cte → Prepares all customer details with balance and city.
- ✓ Main query joins both → Filters customers from 'Blore' whose balance is above their city's average.



## 5.6. Recursive CTE

- Recursive CTEs are used to traverse hierarchical or tree-structured data, such as:
  - ✓ Employee-Manager relationships
  - ✓ Folder structures
  - ✓ Category-Subcategory hierarchies
- They allow you to repeatedly query a dataset until a condition is met.

### Syntax

```
WITH RECURSIVE cte_name AS (  
    SELECT ...      -- Base case (starting point)  
    UNION ALL  
    SELECT ... FROM cte_name WHERE ... -- Recursive part (repeats)  
)  
SELECT * FROM cte_name;
```

- ✓ Base query selects the root/starting records.
- ✓ Recursive query repeatedly joins with the CTE to go deeper.
- ✓ Recursion stops when no new rows are produced.

### Practice Lab – Recursive CTE

#### Step 1: Create Table

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(30),  
    manager_id INT  
);
```



## Step 2: Insert Sample Records

```
INSERT INTO employees VALUES
(1, 'Srinivas', NULL), -- CEO (top-level)
(2, 'Hello', 1),      -- Reports to Srinivas
(3, 'SD', 1),         -- Reports to Srinivas
(4, 'Vas', 2),        -- Reports to Hello
(5, 'aaa', 2),        -- Reports to Hello
(6, 'bbb', 3),        -- Reports to SD
(7, 'ccc', 4);       -- Reports to Vas
```

## Recursive CTE Query

### -- Q1) Display all employees with their reporting hierarchy

```
WITH RECURSIVE emp_hierarchy_cte AS (
    -- Base Case: Top-level employee(s)
    SELECT emp_id, emp_name, manager_id, 1 AS level
    FROM employees
    WHERE manager_id IS NULL

    UNION ALL

    -- Recursive Part: Find subordinates of previous level
    SELECT e.emp_id, e.emp_name, e.manager_id, eh.level + 1
    FROM employees e
    JOIN emp_hierarchy_cte eh ON e.manager_id = eh.emp_id
)
SELECT emp_id, emp_name, manager_id, level
FROM emp_hierarchy_cte
ORDER BY level, manager_id;
```



## How It Works

- ✓ Base Case → Selects the root employee(s) (those with manager\_id IS NULL, i.e., top of hierarchy).
- ✓ Recursive Query → Finds employees whose manager\_id matches the emp\_id from the previous level.
- ✓ Each recursion increases the level by +1, representing hierarchy depth.
- ✓ Query repeats automatically until no more child records exist.
- ✓ Final result lists all employees with their level and manager chain.

## 5.7. Benefits of Using CTEs

| Benefit               | Explanation                                          |
|-----------------------|------------------------------------------------------|
| 1) Readability        | Simplifies complex nested queries                    |
| 2) Reusability        | Can be referenced multiple times within one query    |
| 3) Step-by-step logic | Breaks complex logic into smaller parts              |
| 4) Debugging          | Easier to test intermediate results                  |
| 5) Performance        | Often optimized better than deeply nested subqueries |

## 5.8. Best Practices

- Give meaningful CTE names (cust\_info, city\_avg).
- Keep each CTE focused on one logical task.
- Limit recursive depth to avoid infinite loops.
- Use CTEs over nested subqueries for clarity.
- Always test each CTE independently first.



## 6. Window Functions

- 6.1. What are Window Functions?
- 6.2. Practice Lab - Window Functions:
- 6.3. ROW\_NUMBER()
- 6.4. RANK() and DENSE\_RANK()
- 6.5. NTILE(n)
- 6.6. LAG() & LEAD()
- 6.7. Running Total (Cumulative SUM)
- 6.8. Moving Total & Moving Average (Rolling Windows)
- 6.9. Frame Clauses (ROWS BETWEEN...)
- 6.10. Percentile Functions
- 6.11. FIRST\_VALUE() & LAST\_VALUE()
- 6.12. Running Percentage (% Contribution)



## 6.1. What are Window Functions?

- Window functions perform calculations across a set of table rows that are related to the current row, without grouping the data.
- Unlike GROUP BY, window functions don't collapse rows.

### **Syntax:**

```
function_name (expression)
OVER (
  [PARTITION BY column_name]
  [ORDER BY column_name]
)
```

### **Example Window Functions:**

- ✓ ROW\_NUMBER()
- ✓ RANK()
- ✓ DENSE\_RANK()
- ✓ NTILE(n)
- ✓ SUM(), AVG() (as windowed aggregates)
- ✓ MIN(), MAX() (as windowed aggregates)
- ✓ LAG(),
- ✓ LEAD()
- ✓ FIRST\_VALUE() & LAST\_VALUE()
- etc



## 6.2. Practice Lab - Window Functions:

### A) Create Table – myorders

```
DROP TABLE IF EXISTS myorders;
```

```
CREATE TABLE myorders (  
  order_id    INT PRIMARY KEY,  
  customer_name VARCHAR(50) NOT NULL,  
  region     VARCHAR(20) NOT NULL,  
  product_name VARCHAR(50) NOT NULL,  
  order_date  DATE NOT NULL,  
  quantity   INT NOT NULL,  
  unit_price  DECIMAL(10,2) NOT NULL,  
  total_amount DECIMAL(12,2) NOT NULL  
);
```

### B) Insert Sample Data (30 rows)

```
INSERT INTO myorders  
(order_id, customer_name, region, product_name, order_date, quantity, unit_price,  
total_amount)  
VALUES  
(101,'Sri','East','Laptop Sleeve', '2024-01-05',1,5000.00,5000.00),  
(102,'Vas','West','Wireless Mouse', '2024-01-06',1,3000.00,3000.00),  
(103,'sd','North','Keyboard',      '2024-01-06',1,7000.00,7000.00),  
(104,'ds','South','Stapler',       '2024-01-05',1,3500.00,3500.00),  
(105,'Hello','East','Desk Pad',    '2024-01-06',1,1200.00,1200.00),  
  
(106,'Sri','East','Notebook',      '2024-01-07',1,2000.00,2000.00),  
(107,'Vas','West','Pen Set',        '2024-01-07',1,1000.00,1000.00),  
(108,'sd','North','USB Cable',      '2024-01-08',1,4000.00,4000.00),  
(109,'Hai','West','USB Hub',        '2024-01-06',1,2500.00,2500.00),  
(110,'Sri','East','Backpack',       '2024-01-09',1,8000.00,8000.00),
```



```
(111,'Vas','West','Headphones', '2024-01-10',1,9000.00,9000.00),
(112,'sd','North','Fiction Book', '2024-01-07',1,1500.00,1500.00),
(113,'ds','South','Rice Bag', '2024-01-07',1,2200.00,2200.00),
(114,'Hello','East','Whiteboard', '2024-01-08',1,1800.00,1800.00),
(115,'Vas','West','Office Chair', '2024-01-08',1,6500.00,6500.00),

(116,'sd','North','Desk Lamp', '2024-01-09',1,2500.00,2500.00),
(117,'ds','South','T-Shirt', '2024-01-09',1,3200.00,3200.00),
(118,'Hai','West','Portable Speaker', '2024-01-09',1,4200.00,4200.00),
(119,'Sri','East','Laptop', '2024-01-06',1,55000.00,55000.00),
(120,'Hello','East','Phone Stand', '2024-01-10',1, 900.00, 900.00),

(121,'Hai','West','Desk Organizer', '2024-01-10',1,1300.00,1300.00),
(122,'sd','North','Coffee Powder', '2024-01-11',1, 700.00, 700.00),
(123,'ds','South','Jeans', '2024-01-11',1,5200.00,5200.00),
(124,'Sri','East','External SSD', '2024-01-10',1,8500.00,8500.00),
(125,'Vas','West','Notebook Pack', '2024-01-10',1,1200.00,1200.00),

(126,'Sri','East','Mixer Grinder', '2024-01-12',1,3200.00,3200.00),
(127,'sd','North','Cookbook', '2024-01-13',1,4100.00,4100.00),
(128,'ds','South','Air Fryer', '2024-01-13',1,9000.00,9000.00),
(129,'Hello','East','Bluetooth Keyboard','2024-01-12',1,2600.00,2600.00),
(130,'Hai','West','Bag Pack', '2024-01-12',1,2700.00,2700.00);
```



## 6.3. ROW\_NUMBER()

### A) What Is ROW\_NUMBER()?

- ROW\_NUMBER() is a window function that:
  - Assigns a unique sequential number to each row
  - Numbering is based on the ORDER BY you give inside the window
- Numbering Can be done:
  - On the full result set, or
  - Separately inside groups using PARTITION BY
- Important Note:
  - Numbers are always unique within a partition.
  - There are no gaps unless rows are filtered later.

#### **Syntax:**

```
ROW_NUMBER() OVER (  
    [PARTITION BY column1, column2, ...]  
    ORDER BY sort_column1 [ASC|DESC], sort_column2 ...  
)
```

- PARTITION BY → Optional
  - If you give it → numbering restarts for each group
  - If you don't give it → numbering runs over the entire result set
- ORDER BY → Mandatory (for ROW\_NUMBER)
  - Decides who gets 1, 2, 3, ...



## B) Queries

-- Q1) List all orders with a sequence number based on order\_date (oldest order = 1).

```
SELECT
    order_id, customer_name, region, product_name, order_date, total_amount,
    ROW_NUMBER() OVER (
        ORDER BY order_date, order_id
    ) AS row_num
FROM myorders
ORDER BY order_date, order_id;
```

### How It Works

#### 1) FROM myorders

Take all 30 rows.

#### 2) ORDER BY in the window

ORDER BY order\_date, order\_id inside OVER(...)

SQL sorts rows first by date, then by order\_id if same date.

#### 3) ROW\_NUMBER() OVER(...)

Goes through the sorted rows one by one

First row → row\_num = 1

Second row → row\_num = 2

... and so on till 30.

#### 4) Final ORDER BY

Same ORDER BY in the outer query just to show rows in the same order as numbering.



## -- Q2) Writing Q1) With CTE

```
WITH orders_CTE AS (  
    SELECT  
        order_id, customer_name, region, product_name, order_date, total_amount,  
        ROW_NUMBER() OVER (  
            ORDER BY order_date, order_id  
        ) AS row_num  
    FROM myorders  
)  
SELECT *  
FROM orders_CTE  
ORDER BY order_date, order_id;
```

## -- Q3) Writing Q1) With CTE and Showing only rows 11-20

```
WITH orders_CTE AS (  
    SELECT  
        order_id, customer_name, region, product_name, order_date, total_amount,  
        ROW_NUMBER() OVER (  
            ORDER BY order_date, order_id  
        ) AS row_num  
    FROM myorders  
)  
SELECT *  
FROM orders_CTE  
WHERE row_num BETWEEN 11 AND 20  
ORDER BY row_num;
```

-- Q4) For each region, order the orders by highest total\_amount first,  
-- and assign a row number inside each region.

-- This is Top-N per group pattern.

```
SELECT
  order_id, customer_name, region, product_name, order_date, total_amount,
  ROW_NUMBER() OVER (
    PARTITION BY region
    ORDER BY total_amount DESC
  ) AS row_num_in_region
FROM myorders
ORDER BY region, row_num_in_region;
```

### How It Works

#### **1) Partitioning (GROUP-like behaviour but no collapsing)**

PARTITION BY region creates 4 logical groups:

East orders

West orders

North orders

South orders

#### **2) Ordering inside each partition**

Within each region, rows are ordered by total\_amount DESC

Highest order gets row\_num = 1 in that region

#### **3) ROW\_NUMBER()**

For East region: 1, 2, 3, ... (based on amount)

For West region: 1, 2, 3, ...

For North region: 1, 2, 3, ...

For South region: 1, 2, 3, ...

#### **4) Final ORDER BY region, row\_num\_in\_region**

Groups regions together

Inside each region, you see 1,2,3,... in order

**-- Q5) Writing Q4) With CTE**

```
WITH region_orders_CTE AS (  
    SELECT  
        order_id, customer_name, region, product_name, order_date, total_amount,  
        ROW_NUMBER() OVER (  
            PARTITION BY region  
            ORDER BY total_amount DESC  
        ) AS row_num_in_region  
    FROM myorders  
)  
SELECT *  
FROM region_orders_CTE  
ORDER BY region, row_num_in_region;
```

**-- Q6) Top 3 per region**

```
WITH region_orders_CTE AS (  
    SELECT  
        order_id, customer_name, region, product_name, order_date, total_amount,  
        ROW_NUMBER() OVER (  
            PARTITION BY region  
            ORDER BY total_amount DESC  
        ) AS row_num_in_region  
    FROM myorders  
)  
SELECT *  
FROM region_orders_CTE  
WHERE row_num_in_region <= 3  
ORDER BY region, row_num_in_region;
```



## 6.4. RANK() and DENSE RANK()

### A) What are RANK() and DENSE RANK()?

- Both are window functions used for ranking rows within a partition.
- **a) RANK()**
  - Gives the same rank to rows with same values
  - But skips numbers after a tie
  - **Example:**
    - 1, 1, 3, 4 (rank 2 is skipped)
- **b) DENSE\_RANK()**
  - Gives same rank to ties
  - But does NOT skip the next number.
  - **Example:**
    - 1, 1, 2, 3 (no gaps)
- **When do they restart?**
  - If you use PARTITION BY → ranking restarts for each group
  - If not → ranking runs on the entire result set

#### **Syntax:**

```
RANK() OVER (  
    [PARTITION BY column1, column2 ...]  
    ORDER BY sort_column [ASC|DESC]  
)
```

```
DENSE_RANK() OVER (  
    [PARTITION BY column1, column2 ...]  
    ORDER BY sort_column [ASC|DESC]  
)
```

- ORDER BY → decides who gets rank 1, 2, 3...
- PARTITION BY → optional, used to restart ranking inside groups



## B) Queries

-- Q1) Rank orders by total\_amount (highest first) across the entire table

```
SELECT
    order_id, customer_name, region, product_name, order_date, total_amount,
    RANK() OVER (
        ORDER BY total_amount DESC
    ) AS sale_rank,
    DENSE_RANK() OVER (
        ORDER BY total_amount DESC
    ) AS sale_dense_rank
FROM myorders
ORDER BY sale_rank;
```

### How It Works

#### 1) ORDER BY total\_amount DESC inside window

Largest amount → rank 1

Next largest → rank 2

If two rows have same amount → same rank

#### 2) RANK() behaviour

Ex:

55000, 9000, 9000, 8500 ...

RANK → 1, 2, 2, 4 ...

#### 3) DENSE\_RANK() behaviour

Ex:

55000, 9000, 9000, 8500 ...

DENSE\_RANK → 1, 2, 2, 3 ...

#### 4) Final ORDER BY sale\_rank

Shows ranking clearly.



## -- Q2) Writing Q1) With CTE

```
WITH ranked_orders_CTE AS (  
    SELECT  
        order_id, customer_name, region, product_name, order_date, total_amount,  
        RANK() OVER (  
            ORDER BY total_amount DESC  
        ) AS sale_rank,  
        DENSE_RANK() OVER (  
            ORDER BY total_amount DESC  
        ) AS sale_dense_rank  
    FROM myorders  
)  
SELECT *  
FROM ranked_orders_CTE  
ORDER BY sale_rank;
```

## -- Q3) Top 5 highest orders using ranking

```
WITH ranked_orders_CTE AS (  
    SELECT  
        order_id, customer_name, region, product_name, order_date, total_amount,  
        RANK() OVER (  
            ORDER BY total_amount DESC  
        ) AS sale_rank  
    FROM myorders  
)  
SELECT *  
FROM ranked_orders_CTE  
WHERE sale_rank <= 5  
ORDER BY sale_rank;
```

-- Q4) Rank orders within each region (highest first)

-- Very useful for “Top orders per region” analysis.

```
SELECT
    order_id, customer_name, region, product_name, order_date, total_amount,
    RANK() OVER (
        PARTITION BY region
        ORDER BY total_amount DESC
    ) AS region_rank,
    DENSE_RANK() OVER (
        PARTITION BY region
        ORDER BY total_amount DESC
    ) AS region_dense_rank
FROM myorders
ORDER BY region, region_rank;
```

### How It Works

#### **1) PARTITION BY region**

Creates 4 groups:

East, West, North, South

#### **2) ORDER BY inside each region**

Highest total\_amount → rank 1

Next → rank 2

Ties get same rank.

#### **3) RANK() gaps**

e.g., if South region amounts are:

9000, 9000, 5200, 3200

RANK → 1, 1, 3, 4

#### **4) DENSE\_RANK() no gaps**

e.g., if South region amounts are:

9000, 9000, 5200, 3200

DENSE\_RANK - 1, 1, 2, 3

### -- Q5) Writing Q4) With CTE

```
WITH region_ranked_CTE AS (  
    SELECT  
        order_id, customer_name, region, product_name, order_date, total_amount,  
        RANK() OVER (  
            PARTITION BY region  
            ORDER BY total_amount DESC  
        ) AS region_rank  
    FROM myorders  
)  
SELECT *  
FROM region_ranked_CTE  
ORDER BY region, region_rank;
```

### -- Q6) Top 3 highest orders per region (most common use-case)

```
WITH region_ranked_CTE AS (  
    SELECT  
        order_id, customer_name, region, product_name, order_date, total_amount,  
        RANK() OVER (  
            PARTITION BY region  
            ORDER BY total_amount DESC  
        ) AS region_rank  
    FROM myorders  
)  
SELECT *  
FROM region_ranked_CTE  
WHERE region_rank <= 3  
ORDER BY region, region_rank;
```



## 6.5. NTILE(n)

### A) What is NTILE()?

- NTILE(n) is a window function used to divide rows into **N equal buckets**
- These buckets are also called groups or tiles
- It distributes rows as evenly as possible
- Bucket numbers start from 1 to n
- If rows don't divide evenly, earlier buckets get one extra row
- **Think of it like:**
  - NTILE(4) → divide rows into 4 quartiles
  - NTILE(10) → divide into deciles
  - NTILE(100) → percentiles
- **Used heavily in:**
  - Analytics
  - Ranking customers
  - Quartile classifications
  - Performance categorization
- **When do they restart?**
  - If you use PARTITION BY → ranking restarts for each group
  - If not → ranking runs on the entire result set

#### **Syntax:**

```
NTILE(n) OVER (  
    [PARTITION BY column]  
    ORDER BY sort_column  
)
```

- ORDER BY → decides how rows are assigned to buckets
- PARTITION BY → optional (buckets reset per group)



## B) Queries

-- Q1) Divide all orders into 4 buckets (quartiles) based on total\_amount

```
SELECT
  order_id, customer_name, region, product_name, order_date, total_amount,
  NTILE(4) OVER (
    ORDER BY total_amount DESC
  ) AS amount_quartile
FROM myorders
ORDER BY amount_quartile, total_amount DESC;
```

### How It Works

Assume 30 rows in table:

#### 1) NTILE(4)

→ tries to divide 30 rows into 4 buckets

→  $30 \div 4 = 7.5$

So distribution becomes:

Bucket 1 → 8 rows

Bucket 2 → 8 rows

Bucket 3 → 7 rows

Bucket 4 → 7 rows

#### 2) ORDER BY total\_amount DESC

Highest amounts → bucket 1

Lowest amounts → bucket 4

Example:

| total_amount | bucket |
|--------------|--------|
| 55000        | 1      |
| 9000         | 1      |
| 8500         | 1      |
| ...          | 2      |
| ...          | 3      |
| 700          | 4      |

## -- Q2) NTILE(3) – Divide orders into 3 performance tiers

```
SELECT
    order_id, customer_name, region, total_amount,
    NTILE(3) OVER (
        ORDER BY total_amount DESC
    ) AS performance_tier
FROM myorders
ORDER BY performance_tier, total_amount DESC;
```

### How It Works

Assume 30 rows in table:

#### **For 30 rows:**

Tier 1 → 10 rows

Tier 2 → 10 rows

Tier 3 → 10 rows

#### **Used for:**

High-value orders

Medium-value

Low-value

- Q3) NTILE(3) per region (quartiles inside each region)
- Helpful to compare “top performers inside each region”.

```
SELECT
    order_id, customer_name, region, total_amount,
    NTILE(3) OVER (
        PARTITION BY region
        ORDER BY total_amount DESC
    ) AS region_quartile
FROM myorders
ORDER BY region, region_quartile, total_amount DESC;
```

### **How It Works**

#### **1) PARTITION BY region**

Each region is treated separately

#### **2) ORDER BY total\_amount DESC**

Highest amounts → region\_quartile = 1

Lowest amounts → region\_quartile = 4

If region has (say) 8 rows:

Quartile 1 → 2 rows

Quartile 2 → 2 rows

Quartile 3 → 2 rows

Quartile 4 → 2 rows

#### **3) Final output → neat region-wise quartile ranking.**



## -- Q4) NTILE(10) – Decile distribution

```
SELECT
    order_id,
    customer_name,
    total_amount,
    NTILE(10) OVER (
        ORDER BY total_amount DESC
    ) AS decile
FROM myorders
ORDER BY decile, total_amount DESC;
```



## 6.6. LAG() & LEAD()

### A) What are LAG() and LEAD()?

- These are navigation window functions that let you look at:
  - Previous row's value → LAG()
  - Next row's value → LEAD()
- But the magic is:
  - They look at previous/next within the sorted window
  - They do not require self-joins
  - Used for trend analysis, comparisons, growth pattern, day-over-day changes
- **a) LAG():**
  - Gets the value from previous row (based on ORDER BY).
  - Current row: 2025-01-08 => LAG() → data from 2025-01-07
- **b) LEAD():**
  - Gets the value from next row.
  - Current row: 2025-01-08 => LEAD() → data from 2025-01-09
- **Important notes:**
  - If no previous row → LAG returns NULL
  - If no next row → LEAD returns NULL

#### **Syntax:**

LAG(column, offset, default\_value)  
OVER (PARTITION BY column ORDER BY column)

LEAD(column, offset, default\_value)  
OVER (PARTITION BY column ORDER BY column)

- offset → how many rows back/forward (default = 1)
- default\_value → optional (when NULL appears)
- PARTITION BY creates separate timelines per group
- ORDER BY defines the sequence



## B) Queries

- Q1) Compare each order's total\_amount with previous order (overall timeline)
- Oldest → Newest order

```
SELECT
    order_id, customer_name, order_date, total_amount,
    LAG(total_amount) OVER (
        ORDER BY order_date, order_id
    ) AS prev_amount,
    LEAD(total_amount) OVER (
        ORDER BY order_date, order_id
    ) AS next_amount
FROM myorders
ORDER BY order_date, order_id;
```

### How It Works

Assume 30 rows in table:

- 1) ORDER BY order\_date, order\_id**  
Creates full timeline of all orders.
- 2) LAG(total\_amount)**  
Looks one row back.  
First row → NULL  
Second row → gets amount from first row
- 3) LEAD(total\_amount)**  
Looks one row ahead.  
Last row → NULL
- 4) Use case examples:**  
Compare today's sale with yesterday's  
Check if sales are increasing or decreasing  
Detect spikes or drops



- Q2) Compare each customer's orders (trend per customer)
- Perfect for customer purchase history.

```
SELECT
    customer_name, order_id, order_date, total_amount,
    LAG(total_amount) OVER (
        PARTITION BY customer_name
        ORDER BY order_date
    ) AS prev_order_amount,
    LEAD(total_amount) OVER (
        PARTITION BY customer_name
        ORDER BY order_date
    ) AS next_order_amount
FROM myorders
ORDER BY customer_name, order_date;
```

### How It Works

#### 1) PARTITION BY customer\_name

Each customer gets their own timeline  
LAG/LEAD do NOT cross customers

#### 2) ORDER BY order\_date

Orders sorted oldest → latest

#### 3) Row-by-row navigation inside each customer

For "Sri"  
First order → prev = NULL  
Middle orders → compare properly  
Last order → next = NULL

#### Perfect for:

Customer purchase patterns  
See gaps between purchases  
Finding when customer value dropped/increased

### -- Q3) LAG() with default value (avoid NULLs)

```
SELECT
    order_id, order_date, total_amount,
    LAG(total_amount, 1, 0) OVER (
        ORDER BY order_date
    ) AS prev_amount_default_0
FROM myorders;
```

#### **What happens?**

- 1) First row gets 0 instead of NULL
- 2) Useful for calculations when NULL breaks math

### -- Q4) LEAD() to predict next expected amount

```
SELECT
    order_id, order_date, total_amount,
    LEAD(total_amount, 1, 0) OVER (
        ORDER BY order_date
    ) AS next_order_amount
FROM myorders;
```

### -- Q5) Show difference between current and previous order amount(Overall timeline)

```
SELECT
    order_id, customer_name, order_date, total_amount,
    LAG(total_amount) OVER (
        ORDER BY order_date
    ) AS prev_amount,
    (total_amount - LAG(total_amount) OVER (ORDER BY order_date)) AS
    diff_from_previous
FROM myorders
ORDER BY order_date;
```



## -- Q5 with CTE

```
WITH amount_diff_CTE AS (  
  SELECT  
    order_id, customer_name, order_date, total_amount,  
    LAG(total_amount) OVER (  
      ORDER BY order_date  
    ) AS prev_amount  
  FROM myorders  
  ORDER BY order_date  
)  
SELECT order_id, customer_name, order_date, total_amount, prev_amount, (total_amount -  
prev_amount) as diffs  
FROM amount_diff_CTE  
ORDER BY order_date;
```

## -- Q6) Find customers whose purchase amount increased compared to previous order(Positive Trend)

```
WITH amount_diff_CTE AS (  
  SELECT  
    order_id, customer_name, order_date, total_amount,  
    LAG(total_amount,1,0) OVER (  
      PARTITION BY customer_name  
      ORDER BY order_date,order_id  
    ) AS prev_amount  
  FROM myorders  
)  
SELECT order_id, customer_name, order_date, total_amount, prev_amount, (total_amount -  
prev_amount) as diffs  
FROM amount_diff_CTE  
WHERE total_amount > prev_amount  
ORDER BY customer_name;
```



## -- Q7) Region wise - Negative Trend

```
WITH amount_diff_CTE AS (  
  SELECT  
    order_id, customer_name, region, order_date, total_amount,  
    LAG(total_amount,1,0) OVER (  
      PARTITION BY region  
      ORDER BY order_date,order_id  
    ) AS prev_amount  
  FROM myorders  
)  
SELECT order_id, customer_name, region, order_date, total_amount, prev_amount,  
  (total_amount - prev_amount) as diffs  
FROM amount_diff_CTE  
WHERE total_amount < prev_amount  
ORDER BY region;
```

### Summary

| Function     | Meaning              | Example                   |
|--------------|----------------------|---------------------------|
| LAG(x)       | Previous row value   | Yesterday's sale          |
| LEAD(x)      | Next row value       | Tomorrow's trend          |
| PARTITION BY | Separate timelines   | per customer / per region |
| ORDER BY     | Defines the timeline | date ascending            |



## 6.7. Running Total (Cumulative SUM)

### A) What is a Running Total?

- Running Total (also called Cumulative Sum) is:
  - Sum of all previous rows from the beginning of the dataset up to the current row.
- Characteristics:
  - Always starts from the first row
  - Keeps increasing (monotonic)
  - Window does not slide
  - Used to track cumulative progress over time
- **Examples:**
  - Total sales up to today
  - Cumulative revenue
  - Lifetime value of customer
  - Cumulative website visitors

#### **Syntax:**

```
SUM(column_name) OVER (  
    ORDER BY sort_column  
)
```

- This automatically means:
  - RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW



## B) Queries

### -- Q1) Running Total of all orders (full timeline)

```
SELECT
    order_id, customer_name, order_date, total_amount,
    SUM(total_amount) OVER (
        ORDER BY order_date, order_id
    ) AS running_total
FROM myorders
ORDER BY order_date, order_id;
```

#### How It Works

##### 1) If amounts are:

5000, 3000, 7000, 3500

##### 2) Running total becomes:

5000  
5000+3000 = 8000  
8000+7000 = 15000  
15000+3500 = 18500

### -- Q2) Running Total per region

```
SELECT
    region, order_id, order_date, total_amount,
    SUM(total_amount) OVER (
        PARTITION BY region
        ORDER BY order_date, order_id
    ) AS region_running_total
FROM myorders
ORDER BY region, order_date;
```

# Each region gets its own cumulative total.

### -- Q3) Running Total per customer

```
SELECT
    customer_name, order_id, order_date, total_amount,
    SUM(total_amount) OVER (
        PARTITION BY customer_name
        ORDER BY order_date
    ) AS customer_running_total
FROM myorders
ORDER BY customer_name, order_date;
```

\* Shows customer's lifetime spending pattern.

### -- Q4) Running Total per customer using CTE

```
WITH running_CTE AS (
    SELECT
        order_id, customer_name, order_date, total_amount,
        SUM(total_amount) OVER (
            PARTITION BY customer_name
            ORDER BY order_date
        ) AS running_total
    FROM myorders
)
SELECT *
FROM running_CTE
ORDER BY customer_name, order_date;
```



## 6.8. Moving Total & Moving Average (Rolling Windows)

### A) What are Moving Totals & Moving Averages?

- These are rolling (sliding) window calculations.
- **a) Moving Total (Rolling Sum):**
  - Sum of last N rows only, not entire history.
- **b) Moving Average (Rolling Avg)**
  - Average of last N rows only.
- **Key difference from Running Total:**
  - Running Total grows from start → end
  - Moving Total slides over fixed-size window
- **Why use Moving Windows?**
  - Used for:
    - Trend detection
    - Smoothing fluctuations
    - Forecasting
    - Weekly/monthly rolling metrics
    - Stock/financial analysis

#### Syntax:

```
SUM(column_name) OVER (  
    ORDER BY sort_column  
    ROWS BETWEEN X PRECEDING AND CURRENT ROW  
)
```

```
AVG(column_name) OVER (  
    ORDER BY sort_column  
    ROWS BETWEEN X PRECEDING AND CURRENT ROW  
)
```



## B) Queries

### -- Q1) 3-row Moving Total (rolling sum)

```
SELECT
    order_id, order_date, total_amount,
    SUM(total_amount) OVER (
        ORDER BY order_date, order_id
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ) AS moving_total_3
FROM myorders
ORDER BY order_date, order_id;
```

### -- Q2) 3-row Moving Average

```
WITH moving_CTE AS(
SELECT
    order_id, order_date, total_amount,
    AVG(total_amount) OVER (
        ORDER BY order_date, order_id
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ) AS moving_avg
FROM myorders
ORDER BY order_date
)
SELECT order_id, order_date, total_amount, ROUND(moving_avg,2)
FROM moving_CTE;
```

### -- Q3) Moving Total per region

```
SELECT
    region, order_id, order_date, total_amount,
    SUM(total_amount) OVER (
        PARTITION BY region
        ORDER BY order_date, order_id
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ) AS region_moving_total_3
FROM myorders
ORDER BY region, order_date;
```



## -- Q4) 7-row Moving Total (Weekly trend)

```
SELECT
    order_id, order_date, total_amount,
    SUM(total_amount) OVER (
        ORDER BY order_date
        ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
    ) AS moving_total_7
FROM myorders;
```

## -- Q5) 7-row Moving Total (Weekly trend) - Q4 with CTE

```
WITH weekly_trend_CTE AS (
    SELECT
        order_id, order_date, total_amount,
        SUM(total_amount) OVER (
            ORDER BY order_date, order_id
            ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
        ) AS moving_total_7
    FROM myorders
)
SELECT * FROM weekly_trend_CTE;
```

## -- Q6) Moving Totals & Averages using CTE

```
WITH moving_CTE AS (
    SELECT
        order_id, order_date, total_amount,
        SUM(total_amount) OVER (
            ORDER BY order_date
            ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
        ) AS moving_total_3,
        AVG(total_amount) OVER (
            ORDER BY order_date
            ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
        ) AS moving_avg_3
    FROM myorders
)
SELECT *
FROM moving_CTE
ORDER BY order_date;
```



## 6.9. Frame Clauses (ROWS BETWEEN...)

### A) What are Frame Clauses?

- Frame Clause controls exactly which rows are included in a window function's calculation relative to the current row.
- **Think of it like this:**
  - ORDER BY decides the sequence of rows.
  - FRAME CLAUSE decides how many rows in that sequence each window function should look at.
- **Frame Clauses apply to aggregate window functions like:**
  - SUM
  - AVG
  - MIN
  - MAX
  - COUNT.
- **and also affect functions like:**
  - FIRST\_VALUE
  - LAST\_VALUE
- **Why are Frame Clauses needed?**
  - Because without a frame clause,
    - SQL uses a default window frame based on ORDER BY.
  - Default frame:  

RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  - ✓ This behaves like a running total, not a moving window.
  - To use moving windows (e.g., last 3 rows), we MUST use:  

ROWS BETWEEN X PRECEDING AND CURRENT ROW



## Types of Frame Clauses

1. UNBOUNDED PRECEDING
2. UNBOUNDED FOLLOWING
3. CURRENT ROW
4. N PRECEDING
5. N FOLLOWING

### 1. UNBOUNDED PRECEDING

- Start from the first row of the partition.
- **Meaning:**
  - ✓ From Row 1 → current row
- **Used for:**
  - ✓ Running totals
  - ✓ Cumulative averages

### 2. UNBOUNDED FOLLOWING

- Go until the last row of the partition.
- **Meaning:**
  - ✓ From current row → last row
- Rarely used, but useful for.
  - ✓ backward cumulative totals

### 3. CURRENT ROW

- Only current row is included
- Used for.
  - ✓ Isolated row-based calculations



## 4. N PRECEDING

- Include N rows before the current row.

Example:

ROWS BETWEEN 2 PRECEDING AND CURRENT ROW

- **Meaning → 3 rows:**
  - ✓ Row (current)
  - ✓ Row - 1
  - ✓ Row - 2

## 5. N FOLLOWING

- Include N rows after the current row.

Example:

ROWS BETWEEN CURRENT ROW AND 2 FOLLOWING

- **Meaning → 3 rows:**
  - ✓ Row (current)
  - ✓ Next row
  - ✓ Next-next row



## B) Queries

### -- Q1) Running Total (default frame)

-- Uses UNBOUNDED PRECEDING automatically

```
SELECT
    order_id, order_date, total_amount,
    SUM(total_amount) OVER (
        ORDER BY order_date
    ) AS running_total
FROM myorders
ORDER BY order_date;
```

-- Above is same as Below with the addition of Default Frame

```
SELECT
    order_id, order_date, total_amount,
    SUM(total_amount) OVER (
        ORDER BY order_date
        RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS running_total
FROM myorders
ORDER BY order_date;
```

### -- Q2) Running Total

```
SELECT
    order_id, order_date, total_amount,
    SUM(total_amount) OVER (
        ORDER BY order_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS moving_total_3
FROM myorders
ORDER BY order_date;
```



## -- Q3) 3-row Moving Total

```
SELECT
    order_id, order_date, total_amount,
    SUM(total_amount) OVER (
        ORDER BY order_date
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ) AS moving_total_3
FROM myorders
ORDER BY order_date;
```

## -- Q4) 3-row window centered around current row

```
SELECT
    order_id, order_date, total_amount,
    SUM(total_amount) OVER (
        ORDER BY order_date
        ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING
    ) AS centered_total
FROM myorders
ORDER BY order_date;
```

# Window for Row 3: Row2 + Row3 + Row4

## -- Q5) UNBOUNDED FOLLOWING

### -- Backward cumulative totals:

```
SELECT
    order_id, order_date, total_amount,
    SUM(total_amount) OVER (
        ORDER BY order_date
        ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING
    ) AS backward_running_total
FROM myorders;
```



## -- Q6) Frame Clause with PARTITION BY

```
SELECT
    region, order_id, total_amount,
    SUM(total_amount) OVER (
        PARTITION BY region
        ORDER BY order_date
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ) AS region_moving_total_3
FROM myorders
ORDER BY region, order_date;
```

# Each region gets its own sliding window.



## ROWS vs RANGE — Key Difference

### Practice Lab - ROWS vs RANGE

#### A) Create Table

```
CREATE TABLE orders (  
    order_id INT PRIMARY KEY,  
    order_date DATE,  
    total_amount INT  
);
```

#### B) Insert Sample Data

```
INSERT INTO orders (order_id, order_date, total_amount) VALUES  
(1, '2025-01-01', 100),  
(2, '2025-01-02', 200),  
(3, '2025-01-02', 300),  
(4, '2025-01-03', 400),  
(5, '2025-01-04', 500);
```

| order_id | order_date | total_amount |
|----------|------------|--------------|
| 1        | 2025-01-01 | 100          |
| 2        | 2025-01-02 | 200          |
| 3        | 2025-01-02 | 300          |
| 4        | 2025-01-03 | 400          |
| 5        | 2025-01-04 | 500          |



## -- Case 1: Using ROWS

```
SELECT
    order_id, order_date, total_amount,
    SUM(total_amount) OVER (
        ORDER BY order_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS rows_sum
FROM orders;
```

### How ROWS works?

#### ROWS BETWEEN 2 PRECEDING AND CURRENT ROW

- Move window row by row, strictly counting rows
- Always exactly 3 rows (if available)

#### Output (ROWS)

| order_id | date  | amount | rows_sum |
|----------|-------|--------|----------|
| 1        | 1-Jan | 100    | 100      |
| 2        | 2-Jan | 200    | 300      |
| 3        | 2-Jan | 300    | 600      |
| 4        | 3-Jan | 400    | 1000     |
| 5        | 4-Jan | 500    | 1500     |

- ✓ Always using exact previous 2 rows + current
- ✓ Even if dates are same, each is treated separately



## -- Case 2: Using RANGE

```
SELECT
    order_id, order_date, total_amount,
    SUM(total_amount) OVER (
        ORDER BY order_date
        RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS range_sum
FROM orders;
```

### How RANGE works?

RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW

→ Uses value of ORDER BY, not row count

→ All rows with same ORDER BY value are grouped together

### Output (RANGE)

| order_id | date       | amt | range_sum    |
|----------|------------|-----|--------------|
| 1        | 2025-01-01 | 100 | 100          |
| 2        | 2025-01-02 | 200 | <b>600 ?</b> |
| 3        | 2025-01-02 | 300 | 600          |
| 4        | 2025-01-03 | 400 | 1000         |
| 5        | 2025-01-04 | 500 | 1500         |

✓ RANGE groups same-date rows

✓ It does NOT follow row-by-row movement

### Summary

#### Frame Clause

UNBOUNDED PRECEDING  
UNBOUNDED FOLLOWING  
CURRENT ROW  
N PRECEDING  
N FOLLOWING  
ROWS  
RANGE

#### Meaning

From start of partition  
Until last row  
Current row only  
N rows before  
N rows after  
Row-based, safe, predictable  
Value-based, rarely used



## 6.10. Percentile Functions

### A) What are Percentile Functions?

- Percentile functions tell you how far a value is positioned among all other rows.
- **They are used for:**
  - ✓ Analytics
  - ✓ Data science
  - ✓ Customer segmentation
  - ✓ Ranking distributions
  - ✓ Finding top % performers
- **We cover:**
  - ✓ PERCENT\_RANK()
  - ✓ CUME\_DIST()

### B) What is PERCENT\_RANK()?

- PERCENT\_RANK() gives the relative rank of a row in percentage form.

**Formula:**

$$( \text{rank} - 1 ) / ( \text{total\_rows} - 1 )$$

- **Characteristics:**
  - ✓ Always between 0 and 1
  - ✓ Lowest row → 0
  - ✓ Highest row → 1 (or close to 1 in some cases)
  - ✓ Values in between show relative standing

- **Syntax**

```
PERCENT_RANK() OVER (  
    [PARTITION BY column]  
    ORDER BY sort_column  
)
```



## C) Queries

### -- Q1) Percentile rank of orders by total\_amount

```
SELECT
    order_id, total_amount,
    PERCENT_RANK() OVER (
        ORDER BY total_amount
    ) AS prcent_rank
FROM myorders
ORDER BY prcent_rank, total_amount;
```

#### How it works

##### a) Assume sorted values:

700, 900, 1000, 1200, 1500, 2000, 3000, ...

##### b) Ranks:

RANK → 1,2,3...

##### c) Percent Rank:

$(1-1)/(N-1) = 0$

$(2-1)/(N-1) = \text{small \%}$

...

$(\text{last\_row\_rank}-1)/(N-1) = 1$

## D) What is CUME\_DIST()?

- CUME\_DIST() = Cumulative distribution

#### Formula:

**Number of rows ≤ current value / total rows**

#### ■ Characteristics:

- ✓ Always between 1/total\_rows and 1
- ✓ Monotonically increasing
- ✓ Shows the fraction of rows smaller than or equal to current row



## ■ Syntax

```
CUME_DIST() OVER (  
    [PARTITION BY column]  
    ORDER BY sort_column  
)
```

## E) Queries

### -- Q2) Cumulative distribution of orders by total\_amount

```
SELECT  
    order_id, total_amount,  
    CUME_DIST() OVER (  
        ORDER BY total_amount  
    ) AS cum_dist  
FROM myorders  
ORDER BY cum_dist, total_amount;
```

#### How it works

- ✓ Suppose 30 rows total.  
    If total\_amount = 1200 appears in 3 rows:
- ✓ Rows  $\leq 1200 = 5$  rows
- ✓  $CUME\_DIST = 5 / 30 = 0.1666\dots$



## F) Difference between PERCENT\_RANK() and CUME\_DIST()

| Feature     | PERCENT_RANK()                | CUME_DIST()                          |
|-------------|-------------------------------|--------------------------------------|
| Meaning     | Relative standing             | Percent of rows $\leq$ current       |
| Range       | 0 to 1                        | $>0$ to 1                            |
| Ties        | Same percent rank             | Same cume dist                       |
| Formula     | $(\text{rank} - 1) / (N - 1)$ | $\text{count} \leq \text{value} / N$ |
| Lowest row  | Always 0                      | $1/N$                                |
| Highest row | 1                             | 1                                    |

### Quick example:

#### For amounts:

10, 20, 30, 30, 40

#### PERCENT\_RANK():

10  $\rightarrow (1-1)/(5-1) = 0/4 = 0$

20  $\rightarrow (2-1)/4 = 1/4 = 0.25$

30  $\rightarrow (3-1)/4 = 2/4 = 0.5$

30  $\rightarrow (3-1)/4 = 2/4 = 0.5$

40  $\rightarrow (5-1)/4 = 4/4 = 1$

#### CUME\_DIST():

For 10  $\rightarrow 1 \text{ row} \leq 10 \rightarrow 1/5 = 0.2$

For 20  $\rightarrow 2 \text{ rows} \leq 20 \rightarrow 2/5 = 0.4$

For 30  $\rightarrow 4 \text{ rows} \leq 30 \rightarrow 4/5 = 0.8$

For 30  $\rightarrow 4 \text{ rows} \leq 30 \rightarrow 4/5 = 0.8$

For 40  $\rightarrow 5 \text{ rows} \leq 40 \rightarrow 5/5 = 1$

## G) Percentile Functions per Region

-- Very useful for region-wise segmentation.

### -- Q3) Percent Rank per region

```
WITH percent_rant_CTE AS (  
  SELECT  
    region, order_id, total_amount,  
    PERCENT_RANK() OVER (  
      PARTITION BY region  
      ORDER BY total_amount  
    ) AS region_percent_rank  
  FROM myorders  
  ORDER BY region, region_percent_rank  
)  
SELECT region, order_id, total_amount, ROUND(region_percent_rank,2) as P_RANK  
FROM percent_rant_CTE;
```

### -- Q4) CUME\_DIST per region

```
SELECT  
  region, order_id, total_amount,  
  CUME_DIST() OVER (  
    PARTITION BY region  
    ORDER BY total_amount  
  ) AS region_cume_dist  
FROM myorders  
ORDER BY region, region_cume_dist;
```



## 6.11. FIRST\_VALUE() & LAST\_VALUE()

### A) What are FIRST\_VALUE and LAST\_VALUE?

- These two window functions return:
- **FIRST\_VALUE(column)**
  - ✓ First value in the window frame.
- **LAST\_VALUE(column):**
  - ✓ Last value in the window frame.
- They work based on the ORDER BY inside the window.
- **These are heavily used in:**
  - ✓ Time-series analysis
  - ✓ Comparing current vs first order
  - ✓ Trend analysis
  - ✓ Category-first / category-last values
  - ✓ Identifying growth from first purchase
- **Why are they important?**
  - ✓ Because they allow you to answer questions like:
    - What was the customer's first-ever order amount?
    - What was the first sale in each region?
    - How far has the customer grown since their first purchase?
  - ✓ Without using:
    - Subqueries
    - Self joins
    - Grouping
  - ✓ And Just one window function.

## ■ Syntax

```
FIRST_VALUE(column_name) OVER (  
    [PARTITION BY column]  
    ORDER BY column  
)
```

```
LAST_VALUE(column_name) OVER (  
    [PARTITION BY column]  
    ORDER BY column  
    ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING  
)
```

### Important Note for LAST\_VALUE:

- ✓ By default, it only looks up to the current row, so we MUST set the frame:  
**ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING**
- ✓ Otherwise it will not return the true "last" value.

## B) Queries

### -- Q1) FIRST\_VALUE: Customer's first order amount

```
SELECT  
    customer_name, order_id, order_date, total_amount,  
    FIRST_VALUE(total_amount) OVER (  
        PARTITION BY customer_name  
        ORDER BY order_date  
    ) AS first_order_amount  
FROM myorders  
ORDER BY customer_name, order_date;
```

### How it works

- ✓ For each customer:
- ✓ ORDER BY order\_date → sort oldest → newest
- ✓ FIRST\_VALUE picks the first value in that sorted list
- ✓ Same value is repeated for all rows of that customer



## -- Q2) LAST\_VALUE (with correct frame): Customer's latest order amount

```
SELECT
    customer_name, order_id, order_date, total_amount,
    LAST_VALUE(total_amount) OVER (
        PARTITION BY customer_name
        ORDER BY order_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS last_order_amount
FROM myorders
ORDER BY customer_name, order_date;
```

### Why we need the frame?

- ✓ If we DO NOT write the frame:

ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW

- ✓ LAST\_VALUE will give current row value, not the true last value.

## -- Q3) Compare each customer's current order with first order

```
SELECT
    customer_name, order_id, order_date, total_amount,
    FIRST_VALUE(total_amount) OVER (
        PARTITION BY customer_name
        ORDER BY order_date
    ) AS first_order_amount,
    total_amount -
    FIRST_VALUE(total_amount) OVER (
        PARTITION BY customer_name
        ORDER BY order_date
    ) AS diff_from_first
FROM myorders
ORDER BY customer_name, order_date;
```



## -- Q4) Writing Q3 with CTE

```
WITH orders_with_first AS (  
    SELECT  
        customer_name, order_id, order_date, total_amount,  
        FIRST_VALUE(total_amount) OVER (  
            PARTITION BY customer_name  
            ORDER BY order_date  
        ) AS first_order_amount  
    FROM myorders  
)  
SELECT  
    customer_name, order_id, order_date, total_amount,  
    first_order_amount,  
    total_amount - first_order_amount AS diff_from_first  
FROM orders_with_first  
ORDER BY customer_name, order_date;
```

## -- Q5) Region-wise first and last sale amounts

```
SELECT  
    region, order_id, order_date, total_amount,  
    FIRST_VALUE(total_amount) OVER (  
        PARTITION BY region  
        ORDER BY order_date  
    ) AS region_first_sale,  
    LAST_VALUE(total_amount) OVER (  
        PARTITION BY region  
        ORDER BY order_date  
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING  
    ) AS region_last_sale  
FROM myorders  
ORDER BY region, order_date;
```



## -- Q6)Add trend between first and last order

```
WITH orders_with_first_last AS (  
  SELECT  
    customer_name, order_id, order_date, total_amount,  
    FIRST_VALUE(total_amount) OVER (  
      PARTITION BY customer_name  
      ORDER BY order_date  
      ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING  
    ) AS first_order_amount,  
    LAST_VALUE(total_amount) OVER (  
      PARTITION BY customer_name  
      ORDER BY order_date  
      ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING  
    ) AS last_order_amount  
  FROM myorders  
)  
SELECT  
  customer_name, order_id, order_date, total_amount, first_order_amount,  
  last_order_amount,  
  CASE  
    WHEN last_order_amount > first_order_amount THEN 'Increased'  
    WHEN last_order_amount < first_order_amount THEN 'Decreased'  
    ELSE 'No change'  
  END AS lifetime_trend  
FROM orders_with_first_last  
ORDER BY customer_name, order_date;
```



## 6.12. Running Percentage (% Contribution)

### A) What is Running Percentage?

- Running Percentage (also called Cumulative % Contribution) tells:
  - ✓ **What percentage of the total have we reached up to this row?**
- It builds on:
  - ✓ Running Total
  - ✓ Total Sum of all rows
- These are heavily used in:
  - ✓ Pareto Analysis (80/20 rule)
  - ✓ Sales contribution charts
  - ✓ Performance dashboards
  - ✓ Cumulative revenue contribution
  - ✓ Identifying top customers/products

- **Formula**

**Running % = Running Total / Total of all rows**

- **Syntax**

We need 2 window functions:

**1) Running Total**

`SUM(total_amount) OVER (ORDER BY order_date)`

**2) Total Sum (no ORDER BY)**

`SUM(total_amount) OVER ()`

**Combine:**

`(SUM(total_amount) OVER (ORDER BY order_date)) / (SUM(total_amount) OVER ())`



## B) Queries

### -- Q1) Running Percentage of Total Sales (using CTE)

```
WITH sales_running AS (  
  SELECT  
    order_id, order_date, total_amount,  
    -- 1) Running total by date + order  
    SUM(total_amount) OVER (  
      ORDER BY order_date, order_id  
    ) AS running_total,  
    -- 2) Total sum of all orders  
    SUM(total_amount) OVER () AS total_sum  
  FROM myorders  
)  
  
SELECT  
  order_id, order_date, total_amount, running_total, total_sum,  
  ROUND( (running_total * 100.0) / total_sum, 2 ) AS running_percentage  
FROM sales_running  
ORDER BY order_date, order_id;
```

### How it works

- ✓ Assume total sum = 1,00,000
- ✓ If running totals are:
  - 5000 → 5%
  - 15000 → 15%
  - 35000 → 35%
  - 90000 → 90%
  - ...
- ✓ The percentage keeps increasing until 1 (100%).

## -- Q2) Running % by Region (using CTE)

```
WITH region_sales_running AS (  
    SELECT  
        region, order_id, order_date, total_amount,  
        -- Running total for each region  
        SUM(total_amount) OVER (  
            PARTITION BY region  
            ORDER BY order_date, order_id  
        ) AS region_running_total,  
        -- Total sum per region  
        SUM(total_amount) OVER (PARTITION BY region) AS region_total_sum  
    FROM myorders  
)  
SELECT  
    region, order_id, order_date, total_amount, region_running_total, region_total_sum,  
    ROUND( (region_running_total * 100.0) / region_total_sum, 2 ) AS  
    region_running_percentage  
  
FROM region_sales_running  
ORDER BY region, order_date, order_id;
```

### Use Cases

- ✓ Compare region performance
- ✓ Track cumulative region contribution
- ✓ Identify strong/weak regions

### -- Q3) Running % per Customer (Lifetime Contribution)

```
WITH customer_running AS (  
    SELECT  
        customer_name, order_id, order_date, total_amount,  
        -- Running total for each customer  
        SUM(total_amount) OVER (  
            PARTITION BY customer_name  
            ORDER BY order_date  
        ) AS customer_running_total,  
        -- Total purchase value of each customer  
        SUM(total_amount) OVER (PARTITION BY customer_name) AS customer_total_sum  
    FROM myorders  
)  
SELECT  
    customer_name, order_id, order_date, total_amount, customer_running_total,  
    customer_total_sum,  
    ROUND( (customer_running_total * 100.0) / customer_total_sum, 2 ) AS  
    customer_running_percentage  
  
FROM customer_running  
ORDER BY customer_name, order_date;
```

#### Use Cases

- ✓ Understand customer growth
- ✓ Track purchasing journey
- ✓ Customer loyalty segmentation



## -- Q4) Running Percentage of Sales per Month

```
WITH monthly_sales_running AS (  
  SELECT  
    order_id, order_date, total_amount,  
    DATE_FORMAT(order_date, '%Y-%m') AS order_month,  
    -- Running total inside each month  
    SUM(total_amount) OVER (  
      PARTITION BY DATE_FORMAT(order_date, '%Y-%m')  
      ORDER BY order_date, order_id  
    ) AS month_running_total,  
    -- Total sales for that month  
    SUM(total_amount) OVER (  
      PARTITION BY DATE_FORMAT(order_date, '%Y-%m')  
    ) AS month_total_sum  
  FROM myorders  
)  
SELECT  
  order_month, order_id, order_date, total_amount, month_running_total,  
  month_total_sum,  
  ROUND( (month_running_total * 100.0) / month_total_sum, 2 ) AS  
  month_running_percentage  
FROM monthly_sales_running  
ORDER BY order_month, order_date, order_id;
```

### Use Cases

- ✓ Track progress towards monthly sales targets
- ✓ Compare which months reach 80–90% faster
- ✓ Identify slow months early and push offers/campaigns
- ✓ Support month-on-month performance reviews in sales meetings



-- Q5) Top Contributors: 80% Cumulative Contribution (Pareto 80/20)

-- One of the most common business analytics queries.

```
WITH sales_running AS (  
  SELECT  
    order_id,  
    customer_name,  
    order_date,  
    total_amount,  
    -- Running total sorted by highest value  
    SUM(total_amount) OVER (  
      ORDER BY total_amount DESC  
    ) AS running_total,  
    -- Total revenue  
    SUM(total_amount) OVER () AS total_sum  
  FROM myorders  
)  
SELECT *  
FROM sales_running  
WHERE (running_total * 100.0) / total_sum <= 80  
ORDER BY total_amount DESC;
```

### Use Cases

- ✓ Identify top customers/products contributing most revenue
- ✓ 80/20 Pareto analysis
- ✓ Focus on high-impact segments



## 7. SQL Execution Order

7.1. Written Order vs Actual Execution Order

7.2. Step-by-Step Execution Meaning

7.3. Full Execution Example

7.4. Why Execution Order Matters



## 7.1. Written Order vs Actual Execution Order

- SQL does NOT execute in the order we write.
- SQL Engine follows a hidden internal order.

### **Example query:**

```
SELECT cname, SUM(amount)
FROM customers c
JOIN orders o ON c.cid = o.cid
WHERE amount > 500
GROUP BY cname
HAVING SUM(amount) > 2000
ORDER BY SUM(amount) DESC
LIMIT 5;
```

We write in this order:

SELECT → FROM → JOIN → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT



But SQL executes in this order following Order

FROM → JOIN → WHERE → GROUP BY → HAVING → WINDOW FUNCTIONS → SELECT →  
DISTINCT → UNION/INTERSECT/EXCEPT → ORDER BY → LIMIT

- 1) FROM
- 2) JOIN
- 3) WHERE
- 4) GROUP BY
- 5) HAVING
- 6) WINDOW FUNCTIONS (OVER) — inside SELECT / ORDER BY
- 7) SELECT
- 8) DISTINCT
- 9) UNION / INTERSECT / EXCEPT
- 10) ORDER BY
- 11) LIMIT / OFFSET

## 7.2. Step-by-Step Execution Meaning

### ■ Sample Query

```
SELECT cname, SUM(amount)
FROM customers c
JOIN orders o ON c.cid = o.cid
WHERE amount > 500
GROUP BY cname
HAVING SUM(amount) > 2000
ORDER BY SUM(amount) DESC
LIMIT 5;
```



## 1) FROM

- ✓ Loads base tables.

### Example:

- Take customers table and orders table.

## 2) JOIN

- ✓ Applies JOIN conditions.

### Example:

- Match customers.cid = orders.cid.

## 3) WHERE

- ✓ Row-level filtering before grouping.

### Example:

- WHERE amount > 500

- Cannot use aggregates here — GROUP BY has not executed yet.

## 4) GROUP BY

- ✓ Creates groups.

### Example:

- GROUP BY cname

- One group per customer.

## 5) HAVING

- ✓ Filters groups after grouping.

### Example:

- HAVING SUM(amount) > 2000

- Aggregates allowed
- Non-grouped columns not allowed



## 6) WINDOW FUNCTIONS (OVER)

- ✓ Executed after WHERE/GROUP BY/HAVING,
- ✓ but inside SELECT/ORDER BY evaluation.

### Example:

- SUM(amount) OVER (PARTITION BY cname)

### Why?

- SQL forms groups first
- Then calculates window functions while preparing output columns

- Not allowed in WHERE / GROUP BY / HAVING
- Allowed in SELECT / ORDER BY

## 7) SELECT

- ✓ Final column output.

### Example:

- SELECT cname, SUM(amount) AS total\_amt

- Aliases (like total\_amt)
- → usable only in ORDER BY, not WHERE/GROUP BY/HAVING.

## 8) DISTINCT

- ✓ Removes duplicate rows after SELECT.



## 9) SET OPERATIONS

- ✓ UNION / INTERSECT / EXCEPT
- ✓ Each SELECT executes fully, then sets applied.

## 10) ORDER BY

- ✓ Final sorting.

### Example:

- ORDER BY total\_amt DESC

- SELECT aliases allowed
- Window functions allowed

## 11) LIMIT / OFFSET

- ✓ Applied last.

### Example:

- LIMIT 5

- OFFSET 10

- Gives only the final required rows.



## 7.3. Full Execution Example

-- Q1) Top 3 customers with highest purchase.

```
SELECT cname, SUM(amount) AS total_amt
FROM customers
JOIN orders USING (cid)
GROUP BY cname
ORDER BY total_amt DESC
LIMIT 3;
```

### Internal Execution

- 1) FROM → load both tables
- 2) JOIN → match cid
- 3) WHERE → (none)
- 4) GROUP BY → group per customer
- 5) HAVING → (none)
- 6) WINDOW FUNCTIONS → (none)
- 7) SELECT → cname, SUM(amount)
- 8) DISTINCT → (none)
- 9) UNION / INTERSECT / EXCEPT → (none)
- 10) ORDER BY → total\_amt DESC
- 11) LIMIT → pick top 3 rows



## 7.4. Why Execution Order Matters

- **A) Avoid Errors**
  - ✓ Why we can't use SELECT alias in WHERE?
    - Because WHERE runs before SELECT.
  - ✓ Why aggregate doesn't work in WHERE?
    - Because aggregates are computed at GROUP BY / HAVING stage.
- **B) Write Faster, Correct Queries**
  - ✓ Correct use of WHERE vs HAVING
  - ✓ Correct place for window functions (SELECT / ORDER BY)
- **C) Improve Performance**
  - ✓ Filter early using WHERE
  - ✓ Reduce rows before GROUP BY
  - ✓ Reduce sorting work in ORDER BY



## 8. Optimization & Performance Insights

- 8.1. Query Optimization Basics
- 8.2. How Indexes Work
- 8.3. Query Execution Plan (EXPLAIN)
- 8.4. Performance Optimization Techniques
- 8.5. Intro to Query Caching



## 8.1. Query Optimization Basics

- Optimization means making queries run faster by reducing:
  - ✓ Disk I/O
  - ✓ CPU usage
  - ✓ Number of rows processed
  - ✓ Network transfer
- SQL Optimizer does 80% of the work,
- BUT we must write efficient queries.
- **Most expensive operations (slowest):**
  - ✓ FULL TABLE SCAN
  - ✓ FULL TABLE JOIN
  - ✓ SORT (ORDER BY)
  - ✓ GROUP BY on large tables
  - ✓ DISTINCT on large datasets
- **Our goal = reduce these using:**
  - ✓ Indexes
  - ✓ Filters in WHERE
  - ✓ Proper join conditions
  - ✓ LIMIT
  - ✓ Pre-aggregated tables (optional)



## 8.2. How Indexes Work

- **Indexes = like a book index → fast lookup.**

### **Types of Indexes**

- ✓ There are mainly Two Types of Indexes
  - 1. Clustered Index
  - 2. Non-Clustered Index

### **1. Clustered Index**

- ✓ Data stored physically in sorted order
- ✓ Only ONE clustered index per table
- ✓ Usually on Primary Key (PK)
- ✓ FAST for range scans and equality lookups

### **Example - customers table**

- ✓ Assume customers(cid) is the Primary Key → so it becomes a clustered index.

```
SELECT *  
FROM customers  
WHERE cid BETWEEN 100 AND 200;
```

- ✓ **This is extremely fast**
  - Because rows from cid=100 → cid=200 are stored sequentially on disk.
  - Perfect use-case for clustered index
  - DB reads only the required range
  - No full table scan
- ✓ **When clustered index helps?**
  - PK lookups
  - Range filters on PK
  - Sorting by PK
  - Joins using PK



## 2. Non-Clustered Index

- ✓ Separate index structure → stores pointer to actual row
- ✓ MANY non-clustered indexes can exist
- ✓ Useful for columns frequently used in search conditions

### Example 1: phone

- ✓ **Create Index:**

```
CREATE INDEX idx_customers_phone ON customers(phone);
```

- ✓ **Query using this index:**

```
SELECT cid, cname, phone  
FROM customers  
WHERE phone = '123456';
```

- DB uses idx\_customers\_phone → fast lookup
- Without index → full table scan

### Example 1: city

- ✓ **Create Index:**

```
CREATE INDEX idx_customers_city ON customers(city);
```

- ✓ **Query using this index:**

```
SELECT cid, cname, city  
FROM customers  
WHERE city = 'Bangalore';
```

- DB jumps directly to all Bangalore entries
- Useful for filtering by city



## Where Indexes Help Most

### 1) WHERE filters

- ✓ WHERE phone = '9876543210'
- ✓ WHERE city = 'Bangalore'
- ✓ WHERE cid > 5000

### 2) JOINS

- ✓ customers.cid = orders.cid

### 3) ORDER BY

- ✓ If index matches sorting:  
ORDER BY phone  
ORDER BY city

### 4) GROUP BY

- ✓ GROUP BY city  
If city has index → grouping becomes faster.

## When Indexes Do NOT Help

### 1) Using functions on indexed columns

#### Example:

WHERE LOWER(city) = 'mumbai';

- This breaks index.
- DB can't use index → full scan.



## 2. wildcards

### Case 1: Index CAN be used

#### Example:

WHERE city LIKE 'Bang%'

Meaning → starts with "Bang"

#### Why index works here?

- Because the DB can jump directly to the section of the index where all values starting with "Bang" begin.

- So DB goes:

Start at 'Bang' → read forward → stop when pattern breaks.

- FAST
- INDEX USED
- Range scan

### Case 2: Index CANNOT be used

#### Example:

WHERE city LIKE '%lore'

Meaning → ends with "lore"

#### Why index does NOT work?

- Because DB does NOT know what the starting letters are.
- So DB must do:
  - Full table scan
  - Check each row one by one
  - MATCH last letters
  - Slow



### 3. Very small tables

- ✓ DB chooses a full scan because:
  - Full scan is cheaper than using an index tree

### 4. High-write tables (insert/update/delete)

- ✓ Indexes slow down writes because:
  - DB updates index tree for every write
  - More indexes → slower inserts/updates

**Good rule:**

**Keep indexes only on columns used frequently in WHERE, JOIN, ORDER BY, GROUP BY**

## 8.3. Query Execution Plan (EXPLAIN)

**Use:**

**EXPLAIN SELECT ...**

When you write:

```
EXPLAIN SELECT * FROM myorders WHERE region = 'East';  
EXPLAIN SELECT * FROM myorders WHERE order_id = 101;
```

- ✓ SQL does NOT run the query.
- ✓ It tells you HOW it WOULD run it.

**This helps you know:**

- ✓ Does it use Index?
- ✓ Does it scan entire table?
- ✓ How many rows it expects to process?
- ✓ Which join method it picks?
- ✓ Which step is slowest?



## Exploring the Output Columns

### 1) id

- ✓ Query step number
- ✓ Higher = deeper subquery.

### 2) table

- ✓ Which table is being accessed

### 3) type → ACCESS TYPE (VERY IMPORTANT)

- ✓ This tells you how SQL is accessing the table.

#### ACCESS TYPE Values — Ordered from BEST to WORST

- const (BEST)
- eq\_ref
- ref
- range
- index
- ALL (WORST — FULL TABLE SCAN)

#### a) const (BEST)

- Uses PK or unique index lookup, returns 1 row.

##### Example:

WHERE cid = 10

- Super fast → constant time lookup.

#### b) eq\_ref

- Used for joins on Primary Key or UNIQUE key.

##### Example:

orders.cid = customers.cid

- Best join performance.



## c) ref

- Non-unique index lookup.

### Example:

```
WHERE city = 'Delhi'
```

- Fast, uses index.

## d) range

- Index is used for a range scan.

### Example:

```
WHERE cid BETWEEN 100 AND 500  
WHERE order_date > '2024-01-01'
```

- Still very good.

## e) index

- Scanning full index.
- Better than full table scan because index is smaller than table.

## f) ALL (WORST — FULL TABLE SCAN)

- Reads every row.

### Example:

```
WHERE LOWER(city) = 'delhi'  
WHERE name LIKE '%raj'
```

- Index cannot be used → BAD sign.



## 4) key

- ✓ Which index is used
- ✓ Shows the index chosen by optimizer
- ✓ If NULL → index NOT used

### Example:

key = myindex → Good.

key = NULL → Index not used → slow.

## 5) rows

- ✓ Estimated number of rows scanned
- ✓ Small number → good
- ✓ Huge number → bad

### Example:

rows = 40 → Perfect.

rows = 3500000 → Bad → heavy scan.

## 6) Extra → additional info

Important flags:

- ✓ **Using index**
  - Query satisfied by index alone (covering index) → BEST
- ✓ **Using where**
  - MySQL applied a filter.
- ✓ **Using temporary**
  - Temporary table created → usually slow
  - Seen in GROUP BY or ORDER BY
- ✓ **Using filesort**
  - ORDER BY cannot use index → slow
- ✓ **Using join buffer**
  - Means join is not using index → slow join



## Putting All Together — Example

### Query:

```
SELECT *  
FROM myorders  
WHERE region = 'East';
```

### EXPLAIN Result: (without Index)

| id | table    | type | key  | rows | Extra       |
|----|----------|------|------|------|-------------|
| 1  | myorders | ALL  | Null | 30   | Using where |

- This query works fine on small dataset, but it is not optimized.
  - ✓ Full table scan (ALL)
  - ✓ No index (key = NULL)
  - ✓ Scans all rows (rows = 30)
  - ✓ Filtering happens after reading entire table
- Now Create an index on the WHERE column to fix this.

```
CREATE INDEX myindex on myorders(region);
```

### EXPLAIN Result: (with Index)

| id | table    | type | key     | ref   | rows | Extra |
|----|----------|------|---------|-------|------|-------|
| 1  | myorders | ref  | myindex | const | 10   | Null  |

- **Meaning:**
  - ✓ Using index myindex → good
  - ✓ Access type = ref → good
  - ✓ rows=10 → only 10 rows scanned
  - ✓ This query is optimized.



## BAD Example

```
SELECT *  
FROM myorders  
WHERE LOWER(region) = 'East';
```

EXPLAIN Result:

| id | table    | type | key  | rows      | Extra       |
|----|----------|------|------|-----------|-------------|
| 1  | myorders | ALL  | NULL | 1,200,000 | Using where |

- **Meaning:**

- ✓ Full table scan (ALL)
- ✓ No index used
- ✓ Scanning 1.2 million rows
- ✓ Very Slow

- **Why?**

- ✓ Because using LOWER() breaks index.



```
WITH cust_totals AS (  
    SELECT  
        a.cid, ad.city, SUM(a.bal) AS total_balance, COUNT(*) AS account_count  
    FROM accounts a  
    JOIN address ad ON a.cid = ad.cid  
    GROUP BY a.cid, ad.city  
)  
,  
avg_total AS (  
    SELECT AVG(total_balance) AS avg_bal  
    FROM (  
        SELECT cid, SUM(bal) AS total_balance  
        FROM accounts  
        GROUP BY cid  
    ) t  
)  
SELECT  
    c.cid, c.cname, ct.city, ct.total_balance, ct.account_count  
FROM customers c  
JOIN cust_totals ct ON c.cid = ct.cid  
CROSS JOIN avg_total av  
WHERE ct.total_balance > av.avg_bal  
ORDER BY ct.total_balance DESC  
LIMIT 5;
```

### DO the following Steps

- Run the above Query with Explain
- See the Output ( Better Writedown)
- Create Index

```
ALTER TABLE accounts ADD INDEX idx_accounts_cid (cid);
```

```
ALTER TABLE address ADD INDEX idx_address_cid (cid);
```

- Run the above Query Again with Explain
- See the Output ( Better Writedown)
- Now Compare



## 8.4 Performance Optimization Techniques

- Optimizing SQL means:
  - ✓ reducing the data scanned
  - ✓ reducing the data sorted
  - ✓ reducing the operations performed by the database engine
- Below are the most powerful real-world techniques to make your queries FAST.

### 1) Filter Early (Predicate Pushdown)

- Apply filters in WHERE, not in HAVING (unless using aggregates).

#### **Example:**

- a) HAVING amount > 500; -- Bad
- b) WHERE amount > 500; -- Good

#### **Key Points:**

- ✓ Reduces rows early
- ✓ Reduces GROUP BY cost
- ✓ Improves sorting performance

### 2) Avoid SELECT \*

- Selecting all columns increases:
  - ✓ Disk I/O
  - ✓ Network load
  - ✓ Memory usage

#### **Example:**

- a) SELECT \* FROM orders; -- Bad
- b) SELECT order\_id, order\_date, total\_amount  
FROM orders; -- Good

#### **Key Points:**

- ✓ Faster I/O
- ✓ Index-friendly
- ✓ Helps optimizer

### 3) Reduce Deeply Nested Subqueries

- Nested subqueries create temporary tables → slow.
- Prefer JOINS or CTEs.

- **Bad**

```
SELECT *  
FROM customers  
WHERE cid IN (  
    SELECT cid  
    FROM orders  
    WHERE cid IN (  
        SELECT cid FROM payments  
    )  
);
```

- **Good**

```
WITH paid AS (  
    SELECT DISTINCT cid FROM payments  
)  
SELECT DISTINCT c.cid, c.cname  
FROM customers c  
JOIN orders o ON c.cid = o.cid  
JOIN paid p ON c.cid = p.cid;
```

**Key Points:**

- ✓ Optimizer-friendly
- ✓ Uses indexes correctly



## 4) Use LIMIT on Large Tables

- Perfect for dashboards, logs, admin pages.

### Example:

```
SELECT *  
FROM logs  
ORDER BY created_at DESC  
LIMIT 100;
```

### Key Points:

- ✓ Skips full table scan
- ✓ Only returns what user needs

## 5) Use Covering Indexes

- Covering index satisfies the query without reading table rows.

### Example:

#### Index:

```
CREATE INDEX idx_orders_date_amount  
ON orders(order_date, total_amount);
```

#### Query:

```
SELECT order_date, total_amount  
FROM orders  
WHERE order_date >= '2024-01-01'  
ORDER BY order_date;
```

### Key Points:

- ✓ WHERE uses index
- ✓ ORDER BY uses index
- ✓ SELECT uses indexed columns only
- ✓ FASTEST execution.



## 6) Avoid Functions on Columns in WHERE

- Functions break index usage.

- **Bad**

```
WHERE YEAR(order_date) = 2024;
```

- **Good**

```
WHERE order_date >= '2024-01-01'  
AND order_date < '2025-01-01';
```

**Key Points:**

- ✓ Range scan
- ✓ Index used

## 7) Avoid Implicit Type Conversions

- Type mismatch → full scan.

- **Bad (phone is VARCHAR)**

```
WHERE phone = 9876543210;
```

- **Good**

```
WHERE phone = '9876543210';
```

**Key Points:**

- ✓ Index works

## 8) Avoid ORDER BY on Non-Indexed Columns

- **Bad**

```
ORDER BY total_amount;
```

- **Create index:**

```
CREATE INDEX idx_order_amount ON orders(total_amount);
```

- **Good**

```
ORDER BY total_amount;
```

**Key Points:**

- ✓ Faster sorting
- ✓ No filesort



## 9) Avoid ORDER BY RAND() (Dead Slow)

- Random value generation per row
- Sorting entire table
- Never use in production

## 10) Avoid OR When Possible

- OR breaks index usage.

### - Bad

```
WHERE city = 'Hyd' OR phone = '9000';
```

### - Good

```
SELECT * FROM customers WHERE city = 'Hyd'  
UNION  
SELECT * FROM customers WHERE phone = '9000';
```

#### Key Points:

- ✓ Both use index
- ✓ Faster execution

## 11) Select Only Required Columns During JOIN

- Avoid heavy SELECT \* joins.

### - Bad

```
SELECT *  
FROM t1  
JOIN t2 ON t1.id = t2.id;
```

### - Good

```
SELECT t1.id, t1.name, t2.amount;
```

#### Key Points:

- ✓ Less memory
- ✓ Faster join



## 12) Join Only On Indexed Columns

- JOIN on non-indexed columns activates JOIN BUFFER → slow.
- Ensure both JOIN columns have indexes
- Or create indexes before joining heavy tables

## 13) Avoid DISTINCT When Possible

- DISTINCT usually hides a wrong JOIN.
- Fix JOIN logic first
- Use GROUP BY only when logically correct

## 14) EXISTS vs IN (Very Important)

- Use IN when subquery is small:

```
WHERE cid IN (SELECT cid FROM small_table);
```

- Use EXISTS when subquery is large:

```
WHERE EXISTS (  
    SELECT 1  
    FROM orders o  
    WHERE o.cid = c.cid  
);
```

### Key Points:

- ✓ EXISTS stops at first match
- ✓ IN loads full list
- ✓ EXISTS best for large datasets



## 15) Proper JOIN Ordering (Start with Filtered Table First)

### Best practice:

- Start with the smallest / most filtered table
- Then join larger tables

### Example:

```
SELECT *  
FROM orders o  
JOIN customers c ON o.cid = c.cid  
WHERE o.order_date > '2024-01-01';
```

### Here:

Orders are filtered → fewer rows

Starting with it = FASTEST plan

### Key Points:

- ✓ Reduces intermediate data
- ✓ Improves index usage
- ✓ Faster join algorithms



## 8.5. Intro to Query Caching

- Most databases use query caching to avoid re-executing the same query repeatedly
- Two types of caching generally happen:
  1. Result Set Cache
  2. Execution Plan Cache

### 1. Result Set Cache

- ✓ If the exact same query is run again
  - DB returns the result from cache instead of reprocessing.

### 2. Execution Plan Cache

- ✓ DB stores the parsed & optimized execution plan,
  - So next time it skips optimization phase.

### **When Cache Hit Happens:**

- Cache hit occurs when:
  - ✓ Query text is exactly identical
  - ✓ Same parameters
  - ✓ Same whitespace / comments
  - ✓ Same bindings (in prepared statements)

### **Why Cache Miss Makes Query Slower:**

- When cache is missed, DB must:
  - ✓ Parse query
  - ✓ Validate syntax
  - ✓ Optimize query
  - ✓ Build execution plan
  - ✓ Execute the plan
- Caching avoids steps 1-4 → much faster.